

MODULE 0: BASICS

Contents

1. JAVA.....	1
2. How Java code executes?.....	2
3. OOPS.....	4
4. OOPS vs POPS	5
5. Basics of OOPS.....	6
6. Objects vs Classes	7
7. Java Packages.....	9
8. Keywords in java.....	9
9. Data Types	10
10. Type Casting	11
11. Access Specifier	11
12. Methods	12

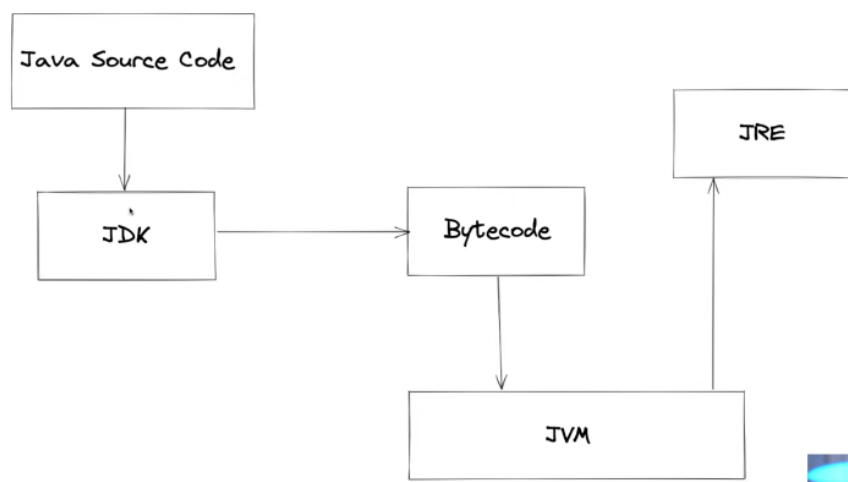
JAVA

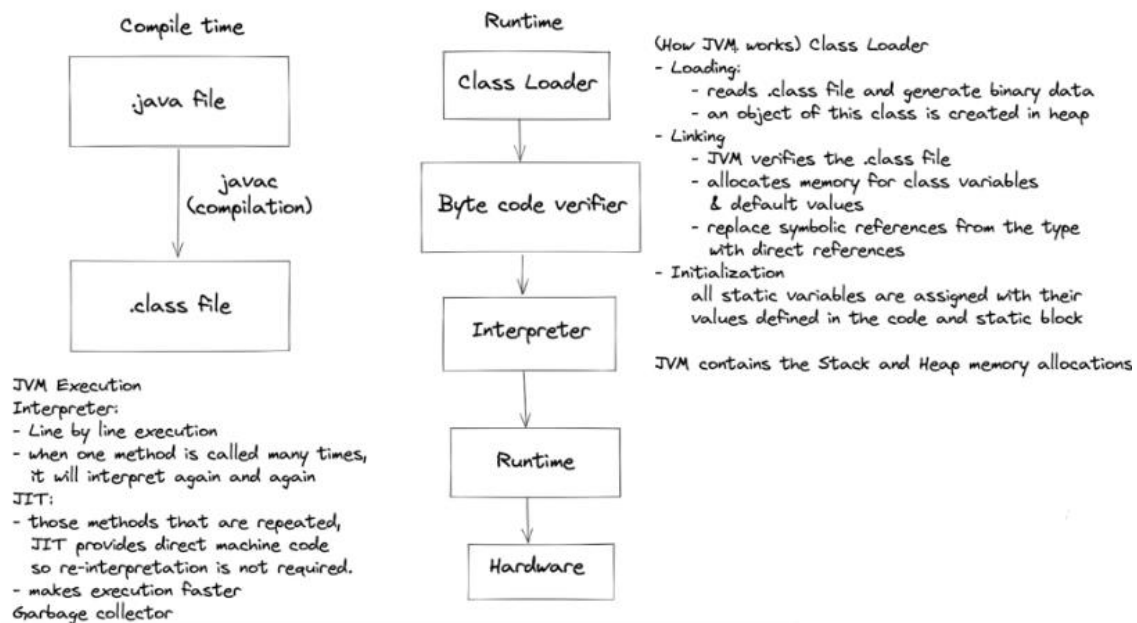
Java is a widely-used, high-level, object-oriented programming language known for its portability, robustness, and security features. Here are its key properties:

1. **Object-Oriented:** Java is designed around the concept of objects. It uses classes and objects to organize code and data. Key object-oriented principles in Java include encapsulation, inheritance, polymorphism, and abstraction. These concepts allow java programs to be more modular, reusable, and maintainable.
2. **Platform Independence:** Java programs are compiled into bytecode, which can be executed on any device equipped with the Java Virtual Machine (JVM). This "write once, run anywhere" capability makes Java platform-independent.
3. **Simple:** Java is designed to be easy to learn and use. It has a clean and straightforward syntax that eliminates many of the complexities of other languages, like C++ (e.g., no pointers, no operator overloading).
4. **Secure:** Java includes robust security features, such as bytecode verification, sandboxing, and the Java security manager, which help protect against malicious code and vulnerabilities.
5. **Robust:** Java emphasizes reliability and robustness. It has strong memory management, exception handling, and type-checking mechanisms, which help prevent runtime errors and make the language more stable.

6. **Multithreaded:** Java supports multithreading, allowing multiple threads of execution within a single program. This enables efficient use of resources and better performance in applications that require parallel processing.
7. **Architecture-Neutral:** Java's bytecode is architecture-neutral, meaning that it does not depend on the underlying hardware or operating system. This ensures that Java applications can run on any platform that supports the JVM.
8. **Interpreted:** Java is both compiled and interpreted. The Java compiler converts source code into bytecode, which is then interpreted by the JVM. This process allows Java programs to be executed on any device with a compatible JVM.
9. **High Performance:** Although Java is an interpreted language, it offers high performance through Just-In-Time (JIT) compilation, where the JVM compiles bytecode to native machine code at runtime, improving execution speed.
10. **Distributed:** Java is designed for the distributed environment of the internet. It includes extensive libraries for networking, making it easier to develop distributed applications.
11. **Dynamic:** Java is a dynamic language that can adapt to an evolving environment. It supports dynamic loading of classes and interfaces, making it easier to integrate with other systems and applications.
12. **Automatic Memory Management (Garbage Collection):** Java automatically manages memory through garbage collection. The JVM automatically identifies and removes objects that are no longer needed, reducing the risk of memory leaks and improving overall application performance.

How Java code executes?

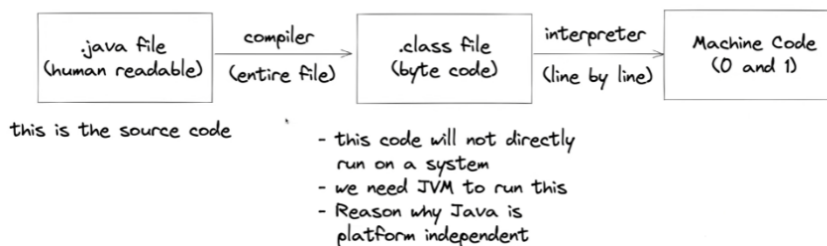




1. **JVM (Java Virtual Machine):** The JVM is a virtual machine that provides a runtime environment to execute Java bytecode. It is platform-independent, meaning the same Java program can run on any device that has a JVM, regardless of the underlying hardware and operating system. **Responsibilities:**

- **Execution of Bytecode:** The JVM reads and executes compiled Java bytecode.
- **Memory Management:** It handles memory allocation and garbage collection.
- **Security:** The JVM includes features to ensure that Java programs are secure, such as bytecode verification and sandboxing.
- **Thread Management:** The JVM manages multiple threads, allowing for concurrent execution within Java programs.

How Java code executes



2. **Byte Code:** Java bytecode is an intermediate language that is generated by compiling a Java program. It is a machine code that runs on the Java virtual machine. It is made up of a series of instructions that tell the JVM how to execute the program. The JVM interprets the bytecode and executes the corresponding instruction.

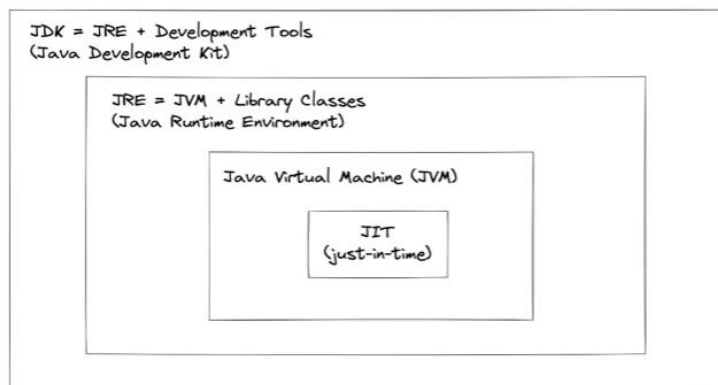
More about platform independence

- It means that byte code can run on all operating systems.
- We need to convert source code to machine code so computer can understand
- Compiler helps in doing this by turning it into executable code
- this executable code is a set of instructions for the computer
- After compiling C/C++ code we get .exe file which is platform dependent
- In Java we get bytecode, JVM converts this to machine code
- Java is platform-independent but JVM is platform dependent

3. **JRE (Java Runtime Environment):** The JRE is a part of the Java Development Kit (JDK) that provides the necessary libraries and resources for running Java applications. It includes the JVM, core libraries, and other components to support the execution of Java programs. **Components:**

- **JVM:** The core component that executes the bytecode.
 - **Class Libraries:** A set of pre-written classes that provide commonly used functionality, such as data structures, networking, and file I/O.
 - **Supporting Files:** Configuration files, properties, and other resources that Java applications might require at runtime.
- **Use:** The JRE is what you need to run Java applications. Developers typically need the JDK, which includes the JRE, to develop Java applications.

JDK vs JRE vs JVM vs JIT



4. **JVM TI (Java Virtual Machine Tool Interface):** JVM TI is a programming interface that provides tools and agents with the ability to inspect and control the state and execution of applications running in the JVM. It is primarily used for debugging, profiling, and monitoring Java applications. **Key Features:**

- **Debugging:** JVM TI allows for the inspection of running programs, setting breakpoints, stepping through code, and examining variables.
- **Profiling:** Tools can use JVM TI to monitor and analyze the performance of Java applications, including CPU usage, memory allocation, and thread activity.
- **Event Notification:** JVM TI can notify tools of various events within the JVM, such as class loading, garbage collection, and exceptions.
- **Usage:** JVM TI is often used by developers to build debugging and profiling tools, such as IDEs (Integrated Development Environments) and performance analyzers.

OOPS

OOPS is a programming paradigm that models real-world entities as objects that have attributes (data) and behaviours (methods) rather than functions and logic.

OOPS focuses on the objects that developers want to manipulate rather than the logic required to manipulate them. Additional benefits of OOPS include code reusability, scalability, and efficiency.

The main objective of OOPS is to bind the data and the functions that operate on them together, so that no other part of the code can access the data except that function.

OOPS vs POPS

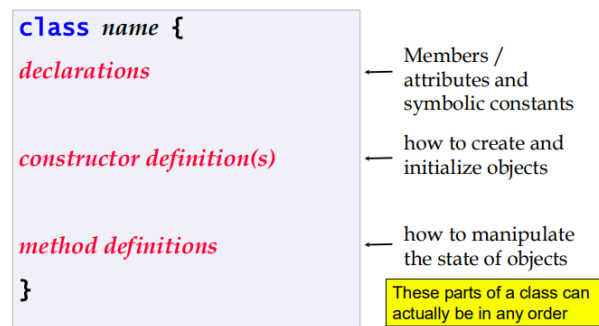
Feature	OOP	POP
Definition	OOPS is a programming paradigm that models real-world entities as objects that have attributes (data) and behaviors (methods)	POPS is a programming paradigm that focuses on procedures or routines (also known as functions) to perform tasks. The program is divided into a set of functions, each performing a specific operation. In POPS, the emphasis is on the sequence of steps (procedures) needed to solve a problem.
Data Encapsulation	Encapsulates data and methods within objects. This encapsulation ensures that data is only accessed through methods, which provides better control and security.	Data and functions are separate. Functions can access and modify data directly, which can lead to less control over data and potential issues with data integrity.
Inheritance, Polymorphism, Abstraction	• Supports inheritance, allowing classes to inherit properties and methods from parent classes. • Supports polymorphism, allowing objects of different classes to be treated as if they were objects of the same class. • Provides a high level of abstraction. Objects represent real-world entities, making the code more intuitive and easier to understand.	• No inheritance • No polymorphism • Provides a lower level of abstraction. The focus is on the process rather than the data, which can make the code harder to understand, especially in complex systems.
Modularity	Promotes high modularity. Each object is a self-contained unit that can be developed and tested independently. This makes code easier to maintain, modify, and extend.	Modularity is achieved by dividing the program into functions, but it's often less modular than OOPS because functions are interdependent and share global data.
Code Reusability	High code reusability through inheritance and polymorphism	Lower code reusability compared to OOP. Functions can be reused, but it's often harder to reuse code across different programs because of the lack of encapsulation and dependence on global data.
Maintainability	Easier to maintain and modify due to modularity and encapsulation	Can be harder to maintain as code grows in complexity

Security	Enhances security by encapsulating data within objects and controlling access through access modifiers (private, public, protected).	Offers less security since data is usually accessible by all functions within the program, leading to potential data corruption.
Example of Use Cases	Used in large, complex applications where modularity, reusability, and real-world modeling are important, such as in enterprise software, game development, and simulation software.	Suitable for smaller, simpler programs where performance and simplicity are key, such as in embedded systems or legacy software.
Example Languages	Java, C++, Python, Ruby, C#	C, Pascal, FORTRAN

Basics of OOPS

- Classes:** A class is a blueprint or template for creating objects. It defines a set of attributes (data) and methods (functions) that the objects created from the class will have. A class encapsulates data for the object and methods to manipulate that data.
- Object:** An object is an instance of a class. It is a concrete entity that has a state (attributes) and behaviour (methods). Objects represent real-world entities and interact with one another in an object-oriented program.
- Data Abstraction:** Data abstraction refers to the concept of hiding the complex implementation details and showing only the essential features of an object. It allows users to interact with the object at a higher level without needing to understand the inner workings.
- Encapsulation:** Encapsulation is the process of bundling data (attributes) and methods (functions) that operate on the data into a single unit, typically a class. It restricts direct access to some of an object's components, which can protect the object's integrity by preventing outside interference and misuse.
- Inheritance:** Inheritance is a mechanism by which one class (known as the subclass or derived class) can inherit properties and behaviours (methods) from another class (known as the superclass or base class). This allows for code reuse and the creation of hierarchical relationships between classes.
- Polymorphism:** Polymorphism is the ability of different classes to be treated as instances of the same class through a common interface. It allows one interface to be used for a general class of actions, with specific behaviour being determined by the exact nature of the class or object.

Structure of a Class Definition



7. **Dynamic Binding:** Dynamic binding, also known as late binding, refers to the process where the method to be executed in response to a method call is determined at runtime rather than at compile-time. It enables polymorphism by allowing objects to decide which method to invoke based on the actual object type at runtime.
8. **Message Passing:** Message passing is the process by which objects communicate with one another. An object sends a message (typically a method call) to another object, requesting it to perform an action or provide some data. This communication is key to interaction in object-oriented programming, allowing objects to work together to accomplish tasks.

Objects vs Classes

Feature	Object	Class
Definition	An object is an instance of a class. It is a concrete entity that has a state (attributes) and behaviour (methods). Objects represent real-world entities and interact with one another in an object-oriented program.	A class is a blueprint or template for creating objects. It defines a set of attributes (data) and methods (functions) that the objects created from the class will have. A class encapsulates data for the object and methods to manipulate that data.
Relationship	Objects are created from classes	Classes define the structure and behaviour of objects
Data	Contains specific values for attributes defined in the class	Defines the attributes that objects will have
Behaviour	Exhibits the methods (functions) defined in the class	Defines the methods that objects will have
Memory allocation	Objects allocate memory space whenever they are created	When a class is created no memory is allocated
As an entity	Object is a physical entity.	Class is a logical entity
Units	Many objects can be created using a class.	We declare class only once
Example	A specific car (e.g., a red Toyota Corolla)	The blueprint for a car, defining its properties and methods

1. **Instance Variable:** An instance variable is a variable that is defined in a class and each object (or instance) of that class has its own copy of the variable. These variables hold the state of an object, meaning they store the data specific to that instance. Instance variables are typically declared within a class but outside any method, constructor, or block. **Example:** If you have a class Car, an instance variable might be color or modelYear, with each Car object having its own specific color or model year.
2. **Attribute of a Class:** An attribute of a class refers to a characteristic or property that describes the objects created from that class. Attributes are essentially the data fields or instance variables in a class. They define what an object knows about itself. **Example:** In a Person class, attributes might include name, age, and address, which store the relevant information for each Person object.

3. **Class Abstraction:** Class abstraction is the concept of defining a class in such a way that it exposes only the essential features needed for interaction while hiding the complex internal details. This allows the class to be used without needing to understand its internal workings fully. **Example:** A BankAccount class might allow users to deposit and withdraw money, but the internal process of updating balances and ensuring security is abstracted away from the user.
4. **Class as an Object Factory:** The concept of a class as an object factory refers to the idea that a class acts as a blueprint or template from which objects are created. The class defines the structure and behaviour, and when an object is instantiated from the class, it is produced or "manufactured" according to that blueprint. **Example:** A Robot class might be designed to produce multiple Robot objects, each with the same basic capabilities but different configurations or behaviours based on the class definition.
5. **Class as a User-Defined Data Type:** A class is considered a user-defined data type because it allows the programmer to create new types that represent complex entities in a program. These data types go beyond the built-in data types like integers or strings, allowing for the modeling of real-world entities. **Example:** When you create a Student class, you are defining a new data type that represents students, with attributes such as name, rollNumber, and methods to manipulate this data.
6. **Class as a Composite Data Type:** A class is a composite data type because it can contain multiple variables (attributes) of different types and methods to operate on those variables. This means that a class can represent complex data structures by combining several pieces of information into one entity. **Example:** A LibraryBook class could include attributes like title, author, ISBN, and checkedOut status, representing the complex nature of a book in a library system.
7. **Object as an Instance of a Class:** An object is an instance of a class, meaning it is a specific realization of the class blueprint. When a class is defined, no memory is allocated until objects of that class are created. Each object is a unique instance with its own set of instance variables (attributes). **Example:** If you have a class Dog, creating an object from this class might look like `Dog myDog = new Dog();`. myDog is now an instance of the Dog class, with its own attributes like name, breed, and age.

Enhanced For Loop

Enhanced For or For-each Style (1D array):

```
for (int x: nums2) sum += x; // start to end
```

Enhanced For or For-each Style (2D array):

```
for (int nums3_1D[] : nums3) // pick up rows
    for (int x : nums3_1D) // pick up elements
        sum += x;
```

Limitations:

- 1) Cannot be used for populating array elements
- 2) Cannot be used for reading alternate array elements unlike classical-for

Java Packages

The Java Development Kit (JDK 1.5) contains Java Class Library that includes various packages. A package is a collection of different classes where each class includes inbuilt functions. Some useful packages in the Java Class Library are as mentioned below:

- `java.lang` : to support classes containing functions to operate on String/character, Math, Integer, Thread, etc.
- `java.io` : to support classes to deal with input and output operation
- `java.applet` : to support classes to generate applets for specific environment
- `java.net` : to support classes for network related operations and dealing with URL (Uniform Resource Locator)
- `java.awt` : to support abstract window tool kit and also to manage GUI (Graphic User Interface)
- `java.text` : to use text elements such as dates, times and currency, etc.
- `java.math` : to support mathematical function such as square roots (integer and decimal both)

Importance of packages

1. **Namespace Management:** Packages provide a way to group related classes and interfaces, avoiding naming conflicts.
2. **Access Control:** Packages help in controlling access to classes, interfaces, and their members using public, protected, and default access modifiers.
3. **Code Organization:** They facilitate better organization of code by categorizing classes and interfaces into coherent modules.
4. **Reusability:** Packages promote reusability of code by allowing classes to be reused across different projects without modification.
5. **Encapsulation:** Packages support encapsulation by hiding the implementation details and exposing only necessary functionalities.
6. **Easier Maintenance:** Well-organized packages make it easier to maintain and update the code, improving overall manageability.
7. **Improved Collaboration:** Packages allow multiple developers to work on different parts of a project without interfering with each other's work.
8. **Library Creation:** They enable the creation of libraries or frameworks that can be easily imported and utilized in various applications.

Keywords in java

1. **“.” Operator:** The dot (.) operator is used to access members (methods and variables) of a class or an object. It is also used to access a package or class from a specific package. **Example:** `myObject.methodName()` accesses the `methodName` method of the `myObject` instance.
2. **new:** The new keyword is used to create new objects in Java. It allocates memory for the object and returns a reference to that memory. **Example:** `MyClass obj = new MyClass();` creates a new instance of `MyClass`.
3. **static:** The static keyword is used to define class-level variables and methods that belong to the class rather than to any specific instance. static members can be accessed directly using the class name. **Example:** `public static int count;` declares a static variable that is shared across all instances of the class.

4. **final:** The final keyword in Java is a modifier that can be applied to variables, methods, and classes. When you declare a primitive data type variable as final, you are making it a constant. This means that once a final variable has been assigned a value, it cannot be reassigned.

- A final variable must be initialized when it is declared, either immediately or within a constructor (for instance variables).
- If it is not initialized at the point of declaration, it must be initialized in every constructor.

```
final int MAX_VALUE = 100;  
MAX_VALUE = 200; // This will cause a compile-time error.
```

1. **const:** The const keyword is reserved in Java but not used. Instead, Java uses the final keyword to define constants. **Example:** final int MAX_VALUE = 100; (not const).
2. **throws:** The throws keyword is used in a method signature to declare that the method might throw exceptions of the specified types. It is used to propagate exceptions to the calling method.
3. **try:** The try keyword is used to define a block of code that will be tested for exceptions during execution. It is part of exception handling and is typically followed by catch or finally blocks.
4. **goto:** The goto keyword is reserved in Java but is not implemented. Java does not support the goto statement as it can lead to unstructured and difficult-to-maintain code. **Example:** Not used in Java.
5. **import:** The import keyword is used to include Java classes or entire packages into your code. This allows the programmer to use the classes defined in other packages. **Example:** import java.util.Scanner; imports the Scanner class from the java.util package.
6. **package:** The package keyword is used to define a package, which is a namespace for organizing classes and interfaces. It helps avoid name conflicts and manage code structure. **Example:** package com.mycompany.myapp; defines a package.
7. **catch:** The catch keyword is used to define a block of code that handles an exception thrown by the try block. Each catch block handles a specific type of exception. They are put for the oops. If some upset difference.

Data Types

Data Type	Description
Primitive Data Types	
byte	8-bit signed integer
short	16-bit signed integer
int	32-bit signed integer
long	64-bit signed integer
float	32-bit floating-point number
double	64-bit floating-point number
char	16-bit Unicode character
boolean	Boolean value (true or false)

Character	ASCII Codes
0-9	48-57
A-Z	65-90
a-z	97-122

Non-Primitive Data Types	
String	Sequence of characters
Array	Ordered collection of elements of the same type
Class	Blueprint for creating objects
Interface	Defines a contract for classes to implement
Enum	Fixed set of constants

Type Casting

1. **Implicit (Automatic) Type Conversion:** occurs automatically when a smaller data type is converted to a larger data type without intervention of user. This is safe because there is no risk of data loss, as the larger type can accommodate all values of the smaller type.
 - **Hierarchy of data types:** the hierarchy shows or indicates the increasing order of data types. If two data of different types are operated, then resulting is automatically get converted to its higher data type.
2. **Explicit (Manual) Type Conversion:** Explicit type conversion, also known as **narrowing conversion** or **casting**, occurs when you explicitly convert a larger data type to a smaller data type. This requires a cast operator because it can lead to data loss if the larger type's value cannot be represented by the smaller type.

byte
char
short
int
long
float
double

Access Specifier

Access specifiers in Java define the visibility and accessibility of classes, methods, and variables in different parts of the program. They control how the members (variables, methods, constructors) of a class can be accessed from other classes. There are four main access specifiers in Java:

1. **public:** When a member is declared as public, it can be accessed from any other class in the same package or any other package. This is the most permissive access level. **Usage:** Use public when you want to allow unrestricted access to a class, method, or variable from anywhere in your application. **Example:**

```
public class MyClass {
    public int myNumber = 10;
    public void display() {
        System.out.println("This is a public method.");
    }
}
```

2. **private:** When a member is declared as private, it can only be accessed within the same class in which it is declared. It is the most restrictive access level. **Usage:** Use private when you want to hide the implementation details and prevent access to the members from outside the class. Typically used to enforce encapsulation. **Example:**

```
public class MyClass {
    private int myNumber = 10;
    private void display() {
        System.out.println("This is a private method.");
    }
}
```

3. **protected:** When a member is declared as protected, it can be accessed within the same package and by subclasses in other packages. It is more restrictive than public but less restrictive than private. **Usage:** Use protected when you want to allow access to class members within the same package or in derived classes (subclasses) even if they are in different packages. **Example:**

```
public class MyClass {
    protected int myNumber = 10;
    protected void display() {
        System.out.println("This is a protected method.");
    }
}
```

4. **Default (Package-Private):** When no access specifier is mentioned, the default access level is applied, which is also known as "package-private" access. Members are accessible only within the same package and not from outside the package. **Usage:** Use default access when you want members to be accessible only to classes within the same package. **Example:**

```
class MyClass {
    int myNumber = 10; // No access specifier, so it's package-private
    void display() {
        System.out.println("This is a package-private method.");
    }
}
```

Summary of Access Levels:

Access Specifier	Same Class	Same Package	Subclass (Different Package)	Other Packages
public	Yes	Yes	Yes	Yes
private	Yes	No	No	No
protected	Yes	Yes	Yes	No
Default	Yes	Yes	No	No

Methods

In Java, **methods** are blocks of code that perform a specific task. They allow you to encapsulate a piece of functionality that can be reused throughout your program. Methods are a fundamental part of object-oriented programming (OOP) as they help organize and modularize code.

1. **Method Declaration:** A method is defined with a name, return type, and a set of parameters (if any). The general syntax for declaring a method is:

```
accessSpecifier returnType methodName(parameters) {  
    // Method body  
}
```

```
public class Example {  
    // A method that returns a string and takes no parameters  
    public String greet() {  
        return "Hello, World!";  
    }  
  
    // A method that takes an integer and returns its square  
    public int square(int number) {  
        return number * number;  
    }  
  
    // A method that does not return any value  
    public void displayMessage() {  
        System.out.println("This is a message.");  
    }  
}
```

2. **Invoking a Method:** Methods are invoked (called) using the method name followed by parentheses. If the method requires parameters, you pass them inside the parentheses.

```
Example example = new Example();  
  
String greeting = example.greet(); // Calling a method that returns a value  
int result = example.square(5);    // Calling a method with a parameter  
example.displayMessage();          // Calling a void method
```

3. Ways of Defining a Method

- i. **Parameterized returnable method:** In this system of defining a method, the method header uses a list of parameters and return type.

Syntax: <Access specifier> <return type> <method name> <parameter list>

```
public int Max(int x)
```

- ii. **Parameterized non-returnable method:** It may happen that the method receives a set of values but does not return the result. In this situation, the method header will use the parameters along with method name, but written type will be void.

Syntax: <Access specifier> <return type> <name> <parameter list>

public void Fact(int x)

- iii. **Nonparameterized non-returnable method:** This type of method header is designed when we neither send any value to the method, nor return any value after execution. The empty braces along with the method name makes it non parameterized.

Syntax: <Access specifier> <return type> <name> <no parameter list>

public void Max()

4. Types of Method Modifiers

In Java, **method modifiers** are keywords that modify the behavior or attributes of methods. Here's a look at the different types of method modifiers, focusing on instance, static, final, and abstract methods:

- a. **Instance Methods:** Instance methods are non-static methods that belong to an instance of a class (an object). These methods require an object to be invoked and can access instance variables and other instance methods of the class.

- Can access both instance variables and static variables.
- Cannot be called without creating an object of the class.
- Can be overridden in a subclass.

```
public class MyClass {  
    int num = 10;  
    public int getNum() {  
        return num; // Instance method  
    }  
}
```

- b. **Static Methods:** static methods belong to the class rather than any specific instance. They can be called without creating an instance of the class and can only directly access other static members (variables and methods) of the class.

- Can be called using the class name directly (e.g., ClassName.methodName()).
- Cannot access instance variables or methods directly; they can only access static variables and methods.
- Cannot be overridden but can be hidden in subclasses.

```
public class MyClass {  
    static int num = 10;  
    public static int getNum() {  
        return num; // Static method  
    }  
}
```

- c. **Abstract Methods:** An abstract method is a method that is declared without an implementation. It is meant to be overridden by subclasses, which are expected to provide a concrete implementation. Abstract methods can only exist in abstract classes or interfaces.

- Does not have a method body; only the method signature is provided.
- Must be implemented by any non-abstract subclass.
- Used to define a common interface for all subclasses, ensuring they implement the method.

```
abstract class Animal {  
    abstract void makeSound(); // Abstract method  
}
```

- d. **Final Methods:** A final method is a method that cannot be overridden by subclasses. This is useful when you want to prevent subclasses from modifying the behaviour of a method.

- Ensures that the method's implementation remains unchanged in any subclass.
- Can be an instance or static method.
- Often used to enforce specific behaviour that should not be altered.

```
public class MyClass {  
    public final void display() {  
        System.out.println("Final method");  
    }  
}
```

5. **Method Overloading:** Method overloading allows a class to have multiple methods with the same name but different parameter lists (i.e., different number of parameters, types of parameters, or both). The correct method to call is determined by the method signature, which includes the method name and parameter types.

- **Same Method Name:** Multiple methods share the same name.
- **Different Parameters:** The methods must differ in the number, type, or order of parameters.
- **Compile-Time Polymorphism:** Method overloading is a form of compile-time polymorphism, meaning that the method to be called is determined at compile time.
- **Return Type:** The return type can be the same or different, but it doesn't play a role in method overloading by itself.

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public double add(double a, double b) {  
        return a + b;  
    }  
  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

6. **Method Overriding:** Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. The method in the subclass must have the same name, return type, and parameters as the method in the superclass.

- **Same Method Signature:** The method in the subclass must have the same name, return type, and parameter list as in the superclass.
- **Runtime Polymorphism:** Method overriding is a form of runtime polymorphism, meaning that the method to be called is determined at runtime based on the object's type.
- **Access Level:** The access level of the overriding method cannot be more restrictive than the method it overrides.

```
class Animal {  
    public void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

- **Annotations:** The `@Override` annotation is often used to indicate that a method is intended to override a method in the superclass. This helps to catch errors at compile time.
- **Super Keyword:** The `super` keyword can be used within the overriding method to call the method from the superclass.

Feature	Method Overloading	Method Overriding
Definition	Multiple methods with the same name but different parameters within the same class.	Subclass provides a specific implementation of a method already defined in its superclass.
Parameters	Must differ in number, type, or order of parameters.	Must have the same parameters as the method in the superclass.
Return Type	Can have the same or different return types.	Must have the same return type (or a covariant return type).
Access Specifier	No restrictions based on the superclass.	Cannot reduce the visibility of the inherited method.
Polymorphism	Compile-time polymorphism (static).	Runtime polymorphism (dynamic).
Inheritance	Does not require inheritance.	Requires inheritance.

7. Pass by Value and Pass by Reference

- Pass by Value:** In Java, all method arguments are passed by value. This means that when a method is called, the actual parameters are passed to the method as a copy of the original values. Changes made to the parameters inside the method do not affect the original values outside the method. **Key Points:**
 - Primitive data types (e.g., `int`, `char`) are passed by value.
 - When passing objects, the reference to the object is passed by value, meaning the reference is copied, but the object itself is not.

```
public class Example {
    public void modifyValue(int number) {
        number = 20; // This change will not affect the original value
    }

    public static void main(String[] args) {
        int num = 10;
        Example ex = new Example();
        ex.modifyValue(num);
        System.out.println(num); // Outputs: 10
    }
}
```

- Pass by Reference:** means passing the actual reference (memory address) of the variable to the method. Any changes made to the parameter affect the original object. Java does not support pass by reference directly, but you can achieve similar behavior by passing objects (not primitive types), as the reference to the object is passed by value. **Key Points:**
 - Changes to the object inside the method will reflect in the original object because both the method parameter and the original reference point to the same object.

```

public class Example {
    public void modifyObject(Demo obj) {
        obj.value = 20; // This change will affect the original object
    }

    public static void main(String[] args) {
        Demo demo = new Demo();
        demo.value = 10;
        Example ex = new Example();
        ex.modifyObject(demo);
        System.out.println(demo.value); // Outputs: 20
    }
}

class Demo {
    int value;
}

```

8. Actual and Formal Parameter

- i. **Actual parameter:** also known as an argument is the value or variable provided to the method when it is called.
 - These are the real values that are passed to the method.
 - They are used in the method call to provide inputs.

```
ex.modifyValue(num); // 'num' is the actual parameter
```

- ii. **Formal Parameter:** are the parameters defined in the method signature. They act as placeholders for the values that will be passed during the method call.
 - Formal parameters are initialized with the values of actual parameters when the method is invoked.
 - They are used within the method to perform operations.

```

public void modifyValue(int number) { // 'number' is the formal parameter
    number = 20;
}

```

9. Pure and Impure Method

- i. **Pure Method:** is a method that, given the same input, will always return the same output and has no side effects (i.e., it does not modify any state or depend on any mutable state).
 - Pure methods are deterministic and do not alter the state of the application.
 - They are often used in functional programming and are easier to test and reason about.

```

public int add(int a, int b) {
    return a + b; // This is a pure method
}

```

- ii. **Impure Method:** is a method that may produce different outputs for the same input or has side effects (i.e., it modifies some state or depends on a mutable state).
 - Impure methods might alter the state of the application, such as modifying an object's field or writing to a file.

- They are more difficult to test and reason about because their behavior can vary.

```
public void incrementCounter() {  
    counter++; // This is an impure method because it modifies the state  
}
```

10. Nested Methods: In Java, methods cannot be nested within each other (i.e., you cannot define a method inside another method directly). However, inner classes (classes defined within another class) can have methods, and they can be used in a similar way to nested methods.

- Java does not support true nested methods, but you can create an inner class with its own methods inside an outer method.
- Some other languages do support nested methods, where a method can be defined within another method.

```
public class OuterClass {  
    public void outerMethod() {  
        class InnerClass {  
            public void innerMethod() {  
                System.out.println("This is a nested method.");  
            }  
        }  
  
        InnerClass inner = new InnerClass();  
        inner.innerMethod(); // Calls the "nested" method  
    }  
  
    public static void main(String[] args) {  
        OuterClass outer = new OuterClass();  
        outer.outerMethod();  
    }  
}
```

MODULE 1: ABSTRACT DATA TYPES

Contents

1. Abstract Data Type (ADT).....	1
2. Concrete State Space	2
3. Concrete Invariant	2
4. Abstraction Function	2
5. Abstract Class.....	3
6. Abstract Method.....	3
7. Abstract Keyword.....	4
8. Interfaces.....	4
9. Interface vs Abstract class.....	5
10. Binding	6
11. Association, Aggregation, Composition	8

Abstract Data Type (ADT)

- An ADT is a mathematical model in data types, that defines a data type by its behavior from the point of view of a user, specifically in terms of the operations that can be performed on it and the mathematical properties of those operations.
- ADTs are abstract because they focus on what the data represents and what operations are available, rather than how these operations are implemented.
- Examples of ADTs: Common examples include stacks, queues, lists, and trees

1.1 Implementation of ADTs

1.1.1 Array ADT

- Access an element: Retrieve an element at a specific index.
- Update an element: Change the value of an element at a specific index.
- Length: Retrieve the number of elements in the array.
- Insert an element: Add an element at a specific position (in the case of dynamic arrays).
- Delete an element: Remove an element at a specific position (in the case of dynamic arrays).

1.1.2 Stack ADT

- push(element): Adds an element to the top of the stack.
- pop(): Removes and returns the top element of the stack.
- peek(): Returns the top element without removing it.
- isEmpty(): Checks if the stack is empty.

1.1.3 Queue ADT

- enqueue(element): Adds an element to the end of the queue.

- `dequeue()`: Removes and returns the element at the front of the queue.
- `peek()`: Returns the element at the front without removing it.
- `isEmpty()`: Checks if the queue is empty.

1.1.4 Tree ADT (Binary Tree):

- `insert(element)`: Adds an element to the tree.
- `find(element)`: Searches for an element in the tree.
- `traverse()`: Visits all elements of the tree in a specific order (e.g., in-order, pre-order, post-order).
- `delete(element)`: Removes an element from the tree.

Concrete State Space

The concrete state space of an ADT implementation refers to the actual data structure and variables used to represent the ADT in memory. It is the set of all possible configurations or values of the data structures that the ADT can hold. Example: For a stack implemented using an array, the concrete state space includes:

- The array that stores the elements.
- A variable (e.g., `topIndex`) that tracks the index of the top element in the stack.
- The size of the array (capacity).

Concrete Invariant

A concrete invariant is a condition or set of conditions that must always hold true for the concrete state space of an ADT implementation. It defines the rules that ensure the data structure remains valid and consistent throughout its operations. Example: For the same stack example, the concrete invariants might be:

- `topIndex >= -1` (the top index is at least -1, indicating an empty stack if it's -1).
- `topIndex < capacity` (the top index is always within the bounds of the array).
- The array elements from 0 to `topIndex` are the elements currently in the stack, while elements beyond `topIndex` are irrelevant or uninitialized.

Importance of Concrete Invariants

- **Ensure Consistency:** They guarantee that the ADT behaves as expected, regardless of the operations performed on it.
- **Facilitate Debugging:** By providing a set of conditions that must always be met, invariants help identify when and where an implementation may be failing.
- **Assist in Verification:** Invariants are used in formal verification methods to prove that an ADT implementation correctly implements its specification.

Abstraction Function

The abstraction function is a crucial concept in the design and implementation of Abstract Data Types (ADTs). It is a way of mapping the concrete representation of an ADT (i.e., the actual data structure and its state in the program) to the abstract view or model of the data type. This mapping helps in reasoning about the correctness and behavior of the ADT by abstracting away the implementation details. Example: Stack ADT

Concrete Representation:

- An array `stack[]` holding the elements.
- An integer `topIndex` indicating the index of the top element.

Abstract Model: A sequence of elements where elements are added and removed in a last-in-first-out (LIFO) manner.

Abstraction Function:

- $AF(stack, topIndex) = [stack[0], stack[1], \dots, stack[topIndex]]$ if $topIndex \geq 0$
- $AF(stack, topIndex) = []$ if $topIndex == -1$

Abstract Class

An abstract class is a class that cannot be instantiated on its own and is meant to be subclassed. It can contain both abstract methods (methods without a body) and concrete methods (methods with an implementation). **Characteristics:**

- Defined using the `abstract` keyword.
- Cannot be instantiated directly (e.g., `new AbstractClass()` is not allowed).
- Can contain abstract methods, concrete methods, fields, and constructors.
- Used to provide a base for subclasses to implement specific behavior.

```
abstract class Animal {  
    abstract void sound(); // Abstract method  
  
    void sleep() { // Concrete method  
        System.out.println("Sleeping...");  
    }  
}
```

Abstract Method

An abstract method is a method that is declared without an implementation (no body) and must be implemented by any concrete subclass of the abstract class. **Characteristics:**

- Defined using the `abstract` keyword.
- Has no body; subclasses must provide an implementation.
- Can only exist in abstract classes or interfaces.

```
abstract class Animal {  
    abstract void sound(); // No implementation provided  
}  
  
class Dog extends Animal {  
    void sound() { // Implementation of the abstract method  
        System.out.println("Bark");  
    }  
}
```

Abstract Keyword

The abstract keyword is used to declare both abstract classes and methods. It signifies that the class or method is incomplete and that it should be further defined by subclasses. **Usage:**

- For **abstract classes**: To indicate that the class cannot be instantiated and may contain abstract methods.
- For **abstract methods**: To indicate that the method must be implemented by subclasses.

Interfaces

In Java, an **interface** is a reference type that is like a class but can only contain method signatures and constant declarations. Interfaces are a core part of Java's object-oriented programming (OOP) paradigm and provide a way to achieve abstraction and multiple inheritance. **Key Features:**

1. **Abstract Methods:**
 - All methods in an interface are abstract by default (i.e., they do not have a body). Classes that implement the interface must provide implementations for these methods.
 - Since Java 8, interfaces can also contain default methods (with a body) and static methods.
2. **Constant Declarations:** Interfaces can declare constants, which are implicitly public, static, and final.
3. **Multiple Inheritance:** A class can implement multiple interfaces, allowing Java to achieve multiple inheritance. This is not possible with classes, as Java does not support multiple inheritance for classes to avoid ambiguity.
4. **Interface Declaration:**
 - An interface is declared using the interface keyword.
 - Interfaces can extend other interfaces, allowing for a hierarchical structure.

Interface Declaration

```
public interface Animal {  
    // Abstract method (no body)  
    void sound();  
  
    // Constant  
    int LEGS = 4; // implicitly public, static, and final  
  
    // Default method (Java 8 and above)  
    default void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

Implementing an Interface

```
public class Dog implements Animal {
    @Override
    public void sound() {
        System.out.println("Bark");
    }

    // Optionally, you can override the default method
    @Override
    public void sleep() {
        System.out.println("Dog sleeping...");
    }
}
```

Advantages of Interfaces

1. **Abstraction:** Interfaces allow you to define a contract that implementing classes must adhere to, providing a clear separation between the specification and implementation.
2. **Multiple Inheritance:** Java allows a class to implement multiple interfaces, enabling a class to inherit behaviors from multiple sources.
3. **Flexibility:** Interfaces enable you to easily swap implementations without changing the code that uses the interface.
4. **Polymorphism:** Interfaces support polymorphic behavior, allowing you to write more generic and reusable code.

Interface vs Abstract class

Aspect	Interface	Abstract Class
Definition	A reference type in Java that can contain constants, method signatures, and default methods. It is a contract that classes must adhere to.	A class that cannot be instantiated and can have abstract methods (without a body) as well as concrete methods (with a body).
Inheritance	Supports multiple inheritance (a class can implement multiple interfaces).	Supports single inheritance (a class can extend only one abstract class).
Method Implementation	Methods are abstract by default (until Java 8 introduced default methods), and cannot have any implementation (except default and static methods).	Can have both abstract methods (without implementation) and concrete methods (with implementation).
Access Modifiers	All methods in an interface are implicitly public and abstract (except static and default methods).	Can have various access modifiers (public, protected, private) for its methods and fields.
Fields	All fields in an interface are implicitly public, static, and final (constants).	Can have instance variables that can be any access modifier (public, protected, private) and can be mutable.

Use Case	Ideal for defining a contract for unrelated classes or behaviors, where multiple classes can implement the same set of methods.	Suitable for providing a common base for related classes with shared behavior and state.
Instantiation	Cannot be instantiated directly; must be implemented by a class.	Cannot be instantiated directly; must be subclassed by a concrete class.
Default Methods	Supports default methods (since Java 8), allowing the interface to provide a default implementation.	Does not support default methods; all methods must be implemented by subclasses unless declared abstract.
Polymorphism	Allows polymorphism through implementation of interfaces.	Allows polymorphism through inheritance of abstract classes.
Example	java interface Animal { void makeSound(); }	java abstract class Animal { abstract void makeSound(); void sleep() { // implementation } }

Binding

In Java, **binding** refers to the linking of a method call to the method body. Binding can be categorized into two types: **static binding** (or early binding) and **dynamic binding** (or late binding). Types:

1. Static Binding (Early Binding): Static binding occurs at compile time. The method to be called is determined based on the reference type (not the object type). This is typically seen with:

- **Final methods:** Methods marked as final cannot be overridden, so the compiler knows which method to call at compile time.
- **Static methods:** Similar to final methods, static methods belong to the class, not to instances of the class.
- **Private methods:** These methods cannot be accessed from outside their class, so their binding is resolved at compile time.

```
class StaticBindingExample {
    public static void display() {
        System.out.println("Static method called.");
    }

    public final void show() {
        System.out.println("Final method called.");
    }
}

public class Main {
    public static void main(String[] args) {
        StaticBindingExample obj = new StaticBindingExample();
        obj.display(); // Static binding
        obj.show();    // Static binding (final method)
    }
}
```

2. Dynamic Binding (Late Binding): Dynamic binding occurs at runtime. The method that is called is determined based on the actual object that the reference points to, not the reference type. This allows for polymorphism, enabling method overriding in subclasses.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound.");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks.");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows.");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog(); // Reference of type Animal, object of type Dog
        Animal myCat = new Cat(); // Reference of type Animal, object of type Cat

        myDog.sound(); // Dynamic binding - outputs: Dog barks.
        myCat.sound(); // Dynamic binding - outputs: Cat meows.
    }
}
```

Key Differences

Feature	Static Binding	Dynamic Binding
Time of Binding	Compile time	Runtime
Method Resolution	Based on reference type	Based on object type
Usage	Final, static, and private methods	Overriding methods in inheritance
Polymorphism	Not applicable	Supports polymorphism

Association, Aggregation, Composition

In Java Object-Oriented Programming (OOP), **association**, **aggregation**, and **composition** represent different types of relationships between classes.

1. Association: A general relationship between two classes where one class uses or interacts with another class. **Example:** A Teacher class and a Student class have an association because teachers teach students, but both can exist independently.

```
class Teacher {
    String name;
    // constructor, getters, and setters
}

class Student {
    String name;
    // constructor, getters, and setters
}

public class Main {
    public static void main(String[] args) {
        Teacher teacher = new Teacher();
        Student student = new Student();
        // Association is implied as the teacher can teach a student, but both can
    }
}
```

// Association is implied as the teacher can teach a student, but both can exist separately.

2. Aggregation: A specialized form of association where one class (the whole) contains a reference to another class (the part), but the part can exist independently of the whole. **Example:** A Department class and a Professor class where Department aggregates Professor. A professor can exist even if the department is dissolved.

```

class Professor {
    String name;
    // constructor, getters, and setters
}

class Department {
    String deptName;
    List<Professor> professors;

    public Department(String deptName, List<Professor> professors) {
        this.deptName = deptName;
        this.professors = professors;
    }
}

public class Main {
    public static void main(String[] args) {
        Professor prof1 = new Professor();
        Professor prof2 = new Professor();
        List<Professor> professorList = Arrays.asList(prof1, prof2);

        Department department = new Department("Computer Science", professorList);
        // Aggregation: Department has professors, but professors can exist without
    }
}

```

// Aggregation: Department has professors, but professors can exist without the department.

3. Composition: A stronger relationship than aggregation where one class (the whole) owns another class (the part), and the part cannot exist without the whole. If the whole is destroyed, the part is also destroyed. **Example:** A Library class and a Book class where Library composes Book. If the library is deleted, the books in it are also deleted.

```
class Book {
    String title;
    // constructor, getters, and setters
}

class Library {
    private List<Book> books = new ArrayList<>();

    public Library() {
        books.add(new Book());
        books.add(new Book());
    }
}

public class Main {
    public static void main(String[] args) {
        Library library = new Library();
        // Composition: Books are part of the library and cannot exist without it.
    }
}
```

MODULE 2: POLYMORPHISM & ENCAPSULATION

Contents

1. Polymorphism	1
2. Bottom of Form Upcasting and Downcasting	3
3. super Keyword	4
4. Polymorphism in Interfaces	6
5. Abstract Classes and Polymorphism	7
6. Advantages of Polymorphism in Object-Oriented Programming (OOP)	8
7. Encapsulation	8
8. Private Access Modifier	9
9. Benefits of encapsulation	10
10. static Keyword in Java	10
11. final Keyword in Java	12
12. Wrapper Classes	13

Polymorphism

Polymorphism is one of the core concepts in Object-Oriented Programming (OOP), which allows an object to take multiple forms. In Java, it enables a single action or method to behave differently based on the object or data type invoking it. Polymorphism promotes flexibility and reusability in code. **Types of Polymorphism:**

1. Compile-Time (Static) Polymorphism: Method Overloading Method overloading occurs when multiple methods in the same class share the same name but have different parameter lists (either by number, type, or order of parameters). The correct method to call is determined at **compile time** based on the method signature (name and parameter list). **Key Points:**

- It is resolved during **compilation**.
- Methods must have the **same name** but **different parameter types** or **different numbers of**

```
Sum of 2 ints: 15
Sum of 2 doubles: 12.0
Sum of 3 ints: 6
```

parameters.

```
class Calculator {
    // Overloaded methods
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class TestOverloading {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        // Calls the method with two integer arguments
        System.out.println("Sum of 2 ints: " + calc.add(5, 10));

        // Calls the method with two double arguments
        System.out.println("Sum of 2 doubles: " + calc.add(5.5, 6.5));

        // Calls the method with three integer arguments
        System.out.println("Sum of 3 ints: " + calc.add(1, 2, 3));
    }
}
```

2. Run-Time (Dynamic) Polymorphism: Method Overriding

Method overriding happens when a **subclass** provides a specific implementation for a method that is already defined in its **superclass**. The version of the method that gets executed is determined at **runtime**, based on the object type. **Key Points:**

- It is resolved during **runtime**.
- The method in the subclass must have the **same name**, **return type**, and **parameter list** as the method in the superclass.
- The overriding method must not have a more restrictive access level than the method being overridden.

Example:

```
Animal makes a sound
Dog barks
Cat meows
```

Here, the `sound()` method is **overridden** in both the `Dog` and `Cat` subclasses, and the method that gets executed depends on the actual object type at runtime, demonstrating **dynamic polymorphism**.

```
class Animal {
    // Method in the superclass
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    // Overriding the sound() method in the subclass
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    // Overriding the sound() method in the subclass
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }
}
```

Aspect	Compile-Time Polymorphism	Run-Time Polymorphism
Definition	Method overloading: Same method name, different parameters.	Method overriding: Subclass provides specific implementation of a method.
Binding Time	Resolved at compile time (early binding).	Resolved at run time (late binding).
Achieved By	Overloading methods.	Overriding methods.
Inheritance Requirement	Not required.	Required.
Performance	Faster due to compile-time resolution.	Slightly slower due to runtime method lookup.
Example	<code>add(int a, int b)</code> and <code>add(float a, float b)</code> .	Overriding <code>toString()</code> in a subclass.
Flexibility	Less flexible since behavior is determined at compile time.	More flexible as behavior can change dynamically at runtime.

Bottom of FormUpcasting and Downcasting

1. Upcasting: Upcasting is the process of converting a **subclass** type to its **superclass** type. It is automatically performed by the compiler, as the subclass is a more specific type, and the superclass is more general.

Why Use It? It allows you to use polymorphism, where a superclass reference can refer to any subclass object. This is useful for writing flexible code that works with superclass types but can handle different subclass objects. **Key Points:**

- Upcasting is **implicit** (automatic) in Java.
- The reference type becomes the **superclass**, but the object is still of the **subclass** type.
- Only the methods available in the **superclass** can be accessed, but overridden methods in the subclass are called.

```
public class TestOverriding {
    public static void main(String[] args) {
        Animal myAnimal = new Animal(); // Animal reference, Animal object
        Animal myDog = new Dog();        // Animal reference, Dog object
        Animal myCat = new Cat();        // Animal reference, Cat object

        // Calls the method based on the object type
        myAnimal.sound(); // Outputs: Animal makes a sound
        myDog.sound();    // Outputs: Dog barks
        myCat.sound();    // Outputs: Cat meows
    }
}

class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class TestUpcasting {
    public static void main(String[] args) {
        Animal myDog = new Dog(); // Upcasting Dog to Animal
        myDog.sound(); // Calls the Dog's overridden method, output: Dog barks
    }
}
```

2. Downcasting: Downcasting is the process of converting a **superclass** type to a **subclass** type. Unlike upcasting, downcasting must be done **explicitly** using casting syntax. **Why Use It?** It is used when you want to access specific methods or properties of a subclass that are not available in the superclass. **Key Points:**

- Downcasting requires **explicit casting** using the () operator.
- It's necessary to **ensure** that the object is actually an instance of the subclass, or a `ClassCastException` will be thrown.

Example:

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    public void bark() {
        System.out.println("Dog barks");
    }
}

public class TestDowncasting {
    public static void main(String[] args) {
        Animal myAnimal = new Dog(); // Upcasting
        ((Dog) myAnimal).bark(); // Downcasting to access Dog-specific method
    }
}
```

super Keyword

The `super` keyword is used to access members (methods or variables) of the **parent class** (superclass) from within a **subclass**. It can be used in three main contexts: calling superclass methods, accessing superclass variables, and calling superclass constructors.

- **`super.method()`:** Used to call an overridden method of the parent class.
- **`super.variable`:** Accesses a hidden variable in the parent class.
- **`super() constructor`:** Invokes the parent class constructor (must be the first line in the subclass constructor).

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    public void sound() {
        super.sound(); // Calls the superclass method
        System.out.println("Dog barks");
    }
}

public class TestSuperMethod {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();
    } // Output: Animal makes a sound, Dog barks
}
```

1. **Accessing Superclass Methods:** The `super` keyword is used to invoke the **overridden method** in the superclass.

2. **Accessing Superclass Variables:** The `super` keyword can be used to reference variables of the superclass if they are **hidden** by the subclass variables.

```
class Animal {
    String name = "Animal";
}
class Dog extends Animal {
    String name = "Dog";
    public void printName() {
        System.out.println(super.name);
        // Access superclass variable
        System.out.println(this.name);
    } // Access subclass variable
}
public class TestSuperVariable {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.printName(); // Output: Animal, Dog
    }
}
```

3. **Calling Superclass Constructors:**
The `super()` constructor call is used to invoke the **parent class constructor**. It must be the **first statement** in the subclass constructor.

Example:

```
class Animal {
    public Animal() {
        System.out.println("Animal constructor");
    }
}
class Dog extends Animal {
    public Dog() {
        super(); // Calls the superclass constructor
        System.out.println("Dog constructor");
    }
}
public class TestSuperConstructor {
    public static void main(String[] args) {
        Dog dog = new Dog();
    } // Output: Animal constructor, Dog constructor
}
```

Polymorphism in Interfaces

Polymorphism in interfaces refers to the ability of a class to implement multiple interfaces, enabling it to exhibit different behaviors depending on the interface used. In Java, interfaces define a contract of methods that any implementing class must provide, promoting loose coupling and flexibility in the system.

- Classes can implement **multiple interfaces** to provide different behavior depending on the interface used.
- Interface references can hold objects of different classes that implement the same interface, enabling **runtime polymorphism**.
- This promotes **flexibility** and **modular design** in systems, as different classes can implement the same interface in their unique ways.

Role of Interfaces in Polymorphism:

- **Multiple Implementations:** A class can implement multiple interfaces, allowing it to take on different forms (polymorphism) based on the interface reference.
- **Method Generalization:** By programming to an interface, you can treat objects of different classes uniformly, provided they implement the same interface. This promotes the **open-closed principle** (OCP) in OOP.

```
interface Animal {
    void sound();
}
interface Flyable {
    void fly();
}
class Bird implements Animal, Flyable {
    public void sound() {
        System.out.println("Bird chirps");
    }
    public void fly() {
        System.out.println("Bird flies");
    }
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class TestInterfacePolymorphism {
    public static void main(String[] args) {
        Animal myAnimal = new Dog();
        // Polymorphism using the Animal interface
        myAnimal.sound(); // Output: Dog barks
        Flyable myBird = new Bird();
        // Polymorphism using the Flyable interface
        myBird.fly(); // Output: Bird flies
    }
}
```

Abstract Classes and Polymorphism

Abstract classes allow polymorphism by providing a common base class with abstract methods that **must be implemented** by any non-abstract subclass. This mechanism forces subclasses to provide specific implementations for the abstract methods, while still allowing shared behavior through non-abstract methods.

- Abstract classes allow **runtime polymorphism** through **method overriding**.
- Subclasses must provide specific implementations for **abstract methods**, but they can also use or override non-abstract methods in the abstract class.
- Abstract classes are particularly useful when you want to provide a common template (shared behavior) but leave specific details to the subclasses.
- Unlike interfaces, abstract classes allow **partial implementation** (having both abstract and concrete methods).

How Abstract Classes Enable Polymorphism:

- **Partial Implementation:** Abstract classes can provide partial method implementations, allowing subclasses to inherit common functionality and override or implement additional behaviors.
- **Runtime Polymorphism:** Abstract classes enable polymorphism because you can use the **abstract class reference** to hold an instance of any subclass, which allows dynamic method dispatch (overridden methods in the subclass are called at runtime).

```
abstract class Animal {  
    // Abstract method (must be implemented by subclasses)  
    abstract void sound();  
  
    // Concrete method (can be inherited or used as is)  
    public void sleep() {  
        System.out.println("Animal sleeps");  
    }  
}  
  
class Dog extends Animal {  
    // Implementing the abstract method  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    // Implementing the abstract method  
    public void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class TestAbstractClassPolymorphism {  
    public static void main(String[] args) {  
        Animal myDog = new Dog(); // Polymorphism using abstract class  
        myDog.sound();             // Output: Dog barks  
        myDog.sleep();             // Output: Animal sleeps  
  
        Animal myCat = new Cat(); // Polymorphism using abstract class  
        myCat.sound();             // Output: Cat meows  
        myCat.sleep();             // Output: Animal sleeps  
    }  
}
```

Advantages of Polymorphism in Object-Oriented Programming (OOP)

1. **Code Reusability:** Polymorphism allows you to reuse code by using a common interface or superclass, reducing redundancy in your programs.
2. **Flexibility:** It provides flexibility by enabling one object to take multiple forms, allowing different implementations of methods depending on the object type at runtime.
3. **Simplified Code Maintenance:** By using polymorphism, you can easily extend or change the behaviour of a class without affecting existing code, improving maintainability.
4. **Loose Coupling:** Polymorphism reduces dependency between classes, promoting loose coupling, which makes your code more modular and easier to manage.
5. **Extensibility:** New classes can be added with minimal changes to existing code, if they follow the same contract (interface or superclass).
6. **Dynamic Method Dispatch:** With polymorphism, the decision of which method to invoke is made at runtime, allowing more dynamic and context-specific behaviour in your programs.
7. **Improved Readability and Clean Design:** Using polymorphism leads to cleaner code with fewer conditionals (like if or switch statements) to check object types, making code more readable.
8. **Easier Code Testing:** Since polymorphic objects are treated in a uniform way, testing different components becomes easier by substituting objects without affecting overall functionality.
9. **Increased Productivity:** Polymorphism enables developers to implement changes and add new features faster due to the modularity and extensibility it offers.
10. **Interface Generalization:** It allows you to create generalized code that works with multiple types, promoting the use of interfaces and abstract classes to define standard behaviour.

Encapsulation

Encapsulation is one of the core principles of Object-Oriented Programming (OOP). It refers to the bundling of data (variables) and methods (functions) that operate on that data into a single unit, typically a class. By doing so, it restricts direct access to some of an object's components and only allows controlled access through well-defined interfaces (methods).

The primary purpose of encapsulation is to hide the internal workings (data and implementation details) of a class from the outside world, exposing only the necessary functionalities. This is achieved by:

- Making the class variables **private**.
- Providing **public getter and setter methods** to access and modify these private variables.

This concept enhances **data security** by preventing unauthorized or accidental modifications to an object's internal state, thereby protecting the integrity of the object. Additionally, encapsulation makes it easier to maintain and update code because changes to the internal implementation don't affect how other parts of the code interact with the object.

```
class Employee {  
    private String name; // Private field  
    private int age;     // Private field  
    // Private fields cannot be accessed  
    // directly outside this class  
}
```

Private Access Modifier

In Java, the private access modifier is used to restrict access to a class's fields and methods. When a class member (such as a variable or method) is declared private, it can only be accessed within the same class. This prevents direct manipulation or retrieval of the member from outside the class, ensuring controlled access and protecting the object's state.

Getter and Setter Methods:

Since the private access modifier prevents direct access to fields from outside the class, **getter** and **setter** methods are introduced to provide controlled access to these private fields. These methods are typically declared as public so that external classes can access and modify the private fields indirectly, ensuring that appropriate validation or constraints can be applied before updating or retrieving the values.

- **Getter:** A method used to retrieve (or "get") the value of a private field.
- **Setter:** A method used to modify (or "set") the value of a private field, often with validation.

```
class Employee {  
    private String name;    // Private field  
    private int age;        // Private field  
  
    // Getter method for 'name'  
    public String getName() {  
        return name;  
    }  
  
    // Setter method for 'name'  
    public void setName(String name) {  
        this.name = name; // Set the value of 'name' field  
    }  
  
    // Getter method for 'age'  
    public int getAge() {  
        return age;  
    }  
  
    // Setter method for 'age' with validation  
    public void setAge(int age) {  
        if(age > 0) {      // Ensuring valid age  
            this.age = age;  
        } else {  
            System.out.println("Invalid age");  
        }  
    }  
}
```

```
public class TestEmployee {  
    public static void main(String[] args) {  
        Employee emp = new Employee();  
  
        // Using setters to set values  
        emp.setName("John");  
        emp.setAge(30);  
  
        // Using getters to retrieve values  
        System.out.println("Employee Name: " + emp.getName());  
        System.out.println("Employee Age: " + emp.getAge());  
    }  
}
```

Benefits of Using Getters and Setters:

1. **Controlled Access:** Provides a layer of control over how class fields are accessed and modified.

2. **Validation:** Enables validation logic before modifying a field. For example, you can prevent setting a negative age by adding conditions in the setter.
3. **Flexibility:** You can change the internal implementation without affecting external code. For instance, you can change how the name is stored (e.g., formatting) without altering the external code using the `getName()` method.
4. **Data Security:** By making fields private and using controlled methods, you enhance the security of sensitive data.

Benefits of encapsulation

1. **Data Hiding:** Encapsulation hides a class's internal implementation, exposing only necessary functions and protecting sensitive data.
2. **Increased Security:** Restricts direct access to an object's state, ensuring data integrity.
3. **Controlled Access:** Uses getter and setter methods for controlled variable access, allowing validation.
4. **Improved Flexibility:** Internal changes don't affect external code, as implementation is hidden.
5. **Easier Maintenance:** Isolates changes within a class, reducing the impact on the rest of the application.
6. **Reduced Complexity:** Simplifies interfaces by hiding complex internal processes.
7. **Enhances Modularity:** Divides the system into manageable units for independent development and testing.
8. **Prevents Data Corruption:** Controls variable access to avoid accidental data modification.
9. **Improved Code Reusability:** Allows classes to be reused without exposing implementation details.
10. **Supports Abstraction:** Lets users focus on what an object does, not how it works, simplifying usage.

static Keyword in Java

The static keyword in Java is used for memory management and can be applied to variables, methods, blocks, and nested classes.

1. **Static Variables (Class Variables):** A static variable is shared among all instances of a class. Instead of each object having its own copy, a single copy of the static variable is created and shared.

```
class Counter {
    static int count = 0; // Static variable
    Counter() {
        count++; // Increment shared count
        System.out.println(count);
    }
}
public class Test {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
    }
}
```

1
2

2. **Static Methods:** Static methods belong to the class, not to objects. These methods can access static variables but **cannot** access non-static instance variables or methods.

```
class MathUtils {
    static int square(int x) { // Static method
        return x * x;
    }
}

public class Test {
    public static void main(String[] args) {
        System.out.println(MathUtils.square(5)); // Call without creating an object
    }
}
```

3. **Static Blocks:** Static blocks are used for static initializations of a class. This block of code runs once when the class is loaded into memory.

```
class Example {
    static {
        System.out.println("Static block executed.");
    }

    Example() {
        System.out.println("Constructor called.");
    }
}

public class Test {
    public static void main(String[] args) {
        Example obj = new Example();
    }
}
```

Static block executed.
Constructor called.

4. **Static Classes:** You can define a static nested class. A static nested class does not have access to instance variables or methods of the outer class.

```
class OuterClass {
    static class StaticNestedClass {
        void display() {
            System.out.println("Static Nested Class");
        }
    }
}

public class Test {
    public static void main(String[] args) {
        OuterClass.StaticNestedClass nestedObj = new OuterClass.StaticNestedClass();
        nestedObj.display();
    }
}
```

final Keyword in Java

The final keyword in Java is used to declare constants or to prevent inheritance and method overriding. It can be applied to variables, methods, and classes.

1. **Final Variables:** A final variable can be assigned only once, making it a constant. Once assigned, its value cannot be changed.

```
class Test {  
    final int MAX = 100;  
    // Final variable (constant)  
    void display() {  
        // MAX = 200;  
        // Error: Cannot assign a value  
        // to final variable 'MAX'  
        System.out.println(MAX);  
    }  
}
```

2. **Final Methods:** A method declared as final cannot be overridden by subclasses.

```
class Parent {  
    final void show() {  
        System.out.println("Final method in Parent");  
    }  
}  
  
class Child extends Parent {  
    // void show() {  
    // Error: Cannot override the final method from Parent  
    //     System.out.println("Attempt to override");  
    // }  
}
```

3. **Final Classes:** A class declared as final cannot be subclassed. This means no class can extend a final class.

```
final class FinalClass {  
    void display() {  
        System.out.println("Final class method");  
    }  
}  
  
// class SubClass extends FinalClass {  
// Error: Cannot subclass a final class  
// }
```

4. **Blank Final Variable:** A final variable can be initialized later but only once. This is called a **blank final variable**.

```
class Test {  
    final int MAX;  
    Test(int max) {  
        MAX = max;  
    } // Final variable  
} // initialized in constructor
```

Wrapper Classes

Wrapper classes in Java are classes that encapsulate (or "wrap") primitive data types into objects. This allows primitives to be treated as objects, which is essential for certain operations, especially in the context of Java's Collections Framework. Each primitive data type has a corresponding wrapper class.

Purpose

- **Object Representation:** Wrapper classes allow primitive data types to be represented as objects. This is necessary for certain data structures and frameworks that require objects, such as collections.
- **Method Compatibility:** They provide methods for converting between different data types and for manipulating primitive values.
- **Enhanced Functionality:** Wrapper classes come with utility methods that enhance the capabilities of primitive types, such as parsing and conversion.

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Key Features of Wrapper Classes:

a. Conversion Between Primitives and Objects:

- **Boxing:** Converting a primitive value into a corresponding wrapper class object.
- **Unboxing:** Converting a wrapper class object back to a primitive value.

```
int num = 10;
Integer wrappedNum = Integer.valueOf(num); // Boxing

int unwrappedNum = wrappedNum.intValue(); // Unboxing
```

b. Autoboxing and Autounboxing:

- Java provides a feature called **autoboxing**, which automatically converts a primitive type into its corresponding wrapper class when needed.
- **Autounboxing** automatically converts a wrapper class object back to its corresponding primitive type.

```
Integer wrappedNum = 10; // Autoboxing: primitive int to Integer object
int num = wrappedNum;    // Autounboxing: Integer object to primitive int
```

c. Methods and Utilities: Wrapper classes provide useful methods to work with the values, such as parsing strings into numbers, comparing values, and more. **Examples:**

- `Integer.parseInt("123")`: Converts a string to an int.
- `Double.toString(10.5)`: Converts a double to a String.
- `Character.isDigit('5')`: Checks if a character is a digit.

- d. **Immutable:** Wrapper class objects are immutable, meaning once a wrapper object is created, its value cannot be changed. If you need a different value, a new wrapper object must be created.
- e. **Constants:** Wrapper classes contain useful constants, such as `Integer.MAX_VALUE`, `Double.POSITIVE_INFINITY`, etc., which represent the limits and special values of the corresponding primitive types.

Example Usage:

```
// Using Integer wrapper class
Integer integerValue = Integer.valueOf(42); // Boxing
int primitiveValue = integerValue.intValue(); // Unboxing

// Autoboxing and Autounboxing in Collections
ArrayList<Integer> list = new ArrayList<>();
list.add(100); // Autoboxing: int to Integer

int num = list.get(0); // Autounboxing: Integer to int
```

Types of Wrapper Classes of 8 primitive data types:

Wrapper Classes	Usage	Example
Integer	Commonly used in collections (e.g., <code>List<Integer></code>).	<code>Integer num = Integer.valueOf(10);</code>
Double	Useful for mathematical operations in collections.	<code>Double num = Double.valueOf(10.5);</code>
Float	Used in floating-point arithmetic with collections	<code>Float num = Float.valueOf(5.5f);</code>
Long	Useful for large integer values in collections	<code>Long num = Long.valueOf(1000000L);</code>
Short	Useful for smaller integer values in collections	<code>Short num = Short.valueOf((short) 10);</code>
Byte	Useful for byte-level data manipulation	<code>Byte num = Byte.valueOf((byte) 1);</code>
Character	Useful for handling character data in collections	<code>Character letter = Character.valueOf('A');</code>
Boolean	Useful for conditions and flags in collections	<code>Boolean flag = Boolean.valueOf(true);</code>

Methods of Wrapper Classes

Wrapper classes in Java come with several utility methods to manipulate and convert primitive data. Some commonly used methods are:

1. **valueOf():** Converts a string or primitive type to the corresponding wrapper object. **Example:** `Integer.valueOf("10");` (returns an `Integer` object).
2. **parseXxx():** Converts a string to the corresponding primitive data type. **Example:** `Integer.parseInt("10");` (returns an `int`).
3. **toString():** Converts the wrapper object into a `String` representation. **Example:** `Integer.toString(10);` (returns the string `"10"`).

4. **xxxValue()**: Returns the value of the wrapper object as its corresponding primitive type. **Example:** Integer obj = 10; obj.intValue(); (returns the int 10).
5. **compareTo()**: Compares two wrapper objects. **Example:** Integer num1 = 5; Integer num2 = 10; num1.compareTo(num2); (returns a negative number because 5 is less than 10).
6. **equals()**: Compares two wrapper objects for equality. **Example:** Integer num1 = 5; Integer num2 = 5; num1.equals(num2); (returns true).
7. **hashCode()**: Returns a hash code value for the wrapper object. **Example:** Integer num = 5; num.hashCode();

Immutable Nature of Wrapper Classes

Wrapper classes in Java are immutable, meaning once an object of a wrapper class is created, its value cannot be changed. If any change is needed, a new wrapper object has to be created. This helps maintain the integrity of data, ensuring values don't get accidentally altered. **Benefits:**

```
Integer num = 10;
num = num + 5;
// A new Integer object is created,
// original 10 remains unchanged
```

- **Thread-Safety:** Immutable objects are inherently thread-safe since their state cannot change after creation.
- **Consistency:** Immutable objects ensure the integrity and consistency of data throughout the program.

Feature	Primitive Data Types	Wrapper Classes
Memory Usage	Use less memory, hold direct values.	Use more memory, represent as objects.
Performance	Generally faster, no object overhead.	Slower due to object creation and boxing/unboxing.
Nullability	Cannot be null, have a default value (e.g., 0).	Can be null, allowing representation of absence of value.
Flexibility	Limited operations (e.g., arithmetic).	Provide utility methods for type conversion, comparison, etc.
Usage in Collections	Cannot be used directly in collections (e.g., List, Set).	Can be used in collections (e.g., List<Integer>, Set<Double>).
Type Safety	Not type-safe in generics (cannot use primitive types).	Enforces type safety in generics (e.g., List<Integer>).
Object-Oriented Features	Cannot leverage OOP features directly.	Can use OOP features like inheritance, polymorphism.
Default Values	Has default values (e.g., 0 for int).	Default value is null when not initialized.
Conversion Methods	No built-in methods for conversion.	Comes with methods (e.g., parseInt(), valueOf()) for conversion.
Thread-Safety	Not inherently thread-safe.	Immutable and thread-safe (once created).

Importance of Wrapper Classes

1. **Integration with Collections:** Wrapper classes allow primitive data types to be used in Java Collections Framework (e.g., List, Set, Map), which only work with objects.
2. **Utility Methods:** They provide various utility methods for converting between types, parsing strings into primitives, and performing comparisons, enhancing the flexibility of data handling.
3. **Object-Oriented Features:** Wrapper classes enable the use of object-oriented principles, allowing developers to leverage features like inheritance, polymorphism, and method overloading.
4. **Nullability:** Unlike primitive types, wrapper objects can represent null, which is useful in scenarios where data may be absent, such as in database operations.
5. **Type Safety in Generics:** Wrapper classes help ensure type safety in generic types, preventing runtime errors by enforcing the correct type at compile time (e.g., List<Integer>).
6. **Compatibility with Streams:** They facilitate seamless integration with Java I/O operations, allowing primitive types to be handled as objects for serialization and deserialization.
7. **Object Representation:** Wrapper classes enable the conversion of literals into object form, which is crucial for advanced operations like sorting and filtering in real-world applications.
8. **Enhanced Functionality:** Wrapper classes support additional functionalities, such as immutability, which contributes to safer and more predictable code.

MODULE 3: INHERITANCE

Contents

Inheritance.....	1
Some Important Concepts of Inheritance	2
Advantages of Inheritance.....	5
Disadvantages of Inheritance	5
Design Patterns.....	6
Iterator Pattern	7

Inheritance

Inheritance in Java is a mechanism where one class (subclass/child class) acquires the properties and behaviors (fields and methods) of another class (superclass/parent class). It promotes code reusability and establishes a parent-child relationship between the classes. By using inheritance, a subclass can extend or modify the existing behavior of a superclass, making the code more modular and easier to maintain.

- The **extends** keyword is used to define inheritance between classes. When one class extends another, the subclass inherits the properties and methods of the superclass.
- **Super Class**: Also known as the **parent class**, it is the class from which other classes inherit. It contains common fields and methods that are shared by all subclasses.
- **Sub Class**: Also known as the **child class**, it is the class that inherits from the super class. The subclass can access and reuse the methods and fields of the superclass. It can also override or extend the superclass's functionality.

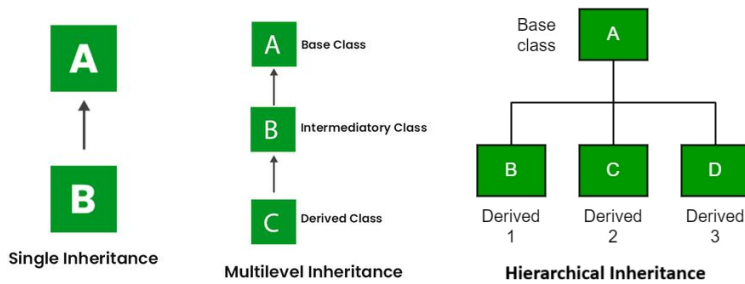
Types of Inheritance in Java

a) Single Inheritance: a subclass inherits from a single superclass. This is the most basic form of inheritance where one class extends another.

```
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
class Dog extends Animal {  
    void bark() { System.out.println("Barking..."); }  
}
```

b) Multilevel Inheritance: a class is derived from another subclass. This forms a chain of inheritance where a subclass further extends a class that has already inherited from another class.

```
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
class Dog extends Animal {  
    void bark() { System.out.println("Barking..."); }  
}  
class Puppy extends Dog {  
    void weep() { System.out.println("Weeping..."); }  
}
```



b) Hierarchical Inheritance: multiple subclasses inherit from the same superclass. This allows different subclasses to share the behavior of a common superclass.

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}
class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}
```

Some Important Concepts of Inheritance

1. Multiple Inheritance (Not Supported in Java):

Java does not support **multiple inheritance** directly, where a class can inherit from more than one class. This limitation avoids ambiguity caused by conflicts from inheriting the same method from multiple classes. However, Java achieves multiple inheritance via **interfaces**.

```
interface A {
    void show();
}
interface B {
    void display();
}
class C implements A, B {
    public void show() {
        System.out.println("Show from A");
    }
    public void display() {
        System.out.println("Display from B");
    }
}
```

2. Code Reusability through Inheritance:

Inheritance promotes **code reusability** by allowing a subclass to inherit properties and methods from a superclass. Instead of writing the same code repeatedly, you can define it once in the superclass and have all subclasses use or override it. This reduces redundancy and improves maintainability.

Here, the Car class reuses the start() method from the Vehicle class without redefining it.

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}
class Car extends Vehicle {
    // Inherits start() from Vehicle
}
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
    } // Reuses start() from Vehicle class
}
```

3. Method Overriding in Inheritance:

Method Overriding occurs when a subclass provides its own implementation of a method that is already defined in the superclass. The overridden method in the subclass has the same signature (method name and parameters) as the method in the superclass. It allows a subclass to customize or extend the behavior of the inherited method.

In this example, Dog overrides the makeSound() method of the Animal class to provide specific behavior.

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
    } // Output: Dog barks (overrides Animal's makeSound)
}
```

4. Super Keyword and its Usage in

Inheritance: The super keyword in Java refers to the **immediate superclass** of the current object. It is used to access superclass methods, fields, and constructors, especially when the subclass has overridden methods or constructors.

- **Accessing Superclass Methods:** The super keyword can be used to call a method from the superclass when it is overridden by the subclass. In this case, super.makeSound() invokes the method from the Animal class, even though Dog overrides it.

In this case, super.makeSound() invokes the method from the Animal class, even though Dog overrides it.

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void makeSound() {
        super.makeSound(); // Calls superclass method
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
    } // Output: Animal makes a sound
    //           Dog barks
}
```

- **Accessing Superclass Constructors:** The super keyword can also be used to call the constructor of the superclass when a subclass constructor is executed. This is useful when the superclass constructor requires parameters.

Here, super("Dog") calls the constructor of the Animal class from the Dog constructor.

```
class Animal {
    Animal(String name) {
        System.out.println("Animal: " + name);
    }
}
class Dog extends Animal {
    Dog() {
        super("Dog"); // Calls the constructor of Animal
        System.out.println("Dog created");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog(); // Output: Animal: Dog
                                //           Dog created
    }
}
```

5. Constructor Chaining: refers to the process of invoking one constructor from another within the same class or across different classes using `this()` or `super()`.

- **In the same class**, constructor chaining is done using the `this()` keyword to call another constructor.
- **In inheritance**, constructor chaining is done using `super()` to call the parent class's constructor.
- The chain starts from the current class constructor and proceeds upwards through the class hierarchy. Every subclass constructor implicitly or explicitly calls the constructor of its superclass.

```
class Animal {
    Animal() {
        System.out.println("Animal is created");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Calls superclass constructor
        System.out.println("Dog is created");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog(); // Output: Animal is created
                           //          Dog is created
    }
}
```

- In this example, when the `Dog` constructor is called, the `Animal` constructor is also invoked because of `super()`. This ensures that the object is initialized from the top of the hierarchy downwards.

6. Final Keyword in Inheritance

- **Preventing Inheritance with final Class:** When a class is declared with the `final` keyword, it cannot be inherited by any subclass. For example:

```
final class Animal { }
class Dog extends Animal { }
// This will give a compile-time error.
```

- **Preventing Method Overriding with final Methods:** A method declared as `final` cannot be overridden by any subclass. For example:

```
class Animal {
    final void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void makeSound() { // This will give a compile-time error.
        System.out.println("Dog barks");
    }
}
```

7. Polymorphism in Inheritance:

Polymorphism allows the subclass to define its own behavior while inheriting the parent class's properties. A parent class reference can refer to a child class object, providing flexible code that works with different types of objects.

```
Animal animal = new Dog();
// Animal reference, Dog object.
animal.makeSound();
// Calls Dog's overridden method.
```

8. Upcasting and Downcasting in Inheritance

- **Upcasting:** Casting a subclass object to its superclass type. This is done automatically in Java.
- **Downcasting:** Casting a superclass reference back to a subclass type. It needs to be done manually and checked at runtime to avoid `ClassCastException`.

```
Animal animal = new Dog();
// Upcasting
Dog dog = (Dog) animal;
// Downcasting
```

Advantages of Inheritance

1. **Code Reusability:** Allows reuse of existing code, reducing redundancy by inheriting fields and methods from a superclass.
2. **Method Overriding:** Enables subclass-specific implementations of methods, enhancing flexibility.
3. **Polymorphism:** Facilitates dynamic method dispatch, allowing different implementations for a single method call.
4. **Simplifies Maintenance:** Changes made in the parent class are reflected in all child classes, making maintenance easier.
5. **Encourages Modular Design:** Promotes cleaner and more modular code organization through hierarchical structures.
6. **Supports Extensibility:** Enables adding new functionalities to existing code without modifying it directly.
7. **Consistency:** Helps maintain consistency across similar types of objects by sharing common functionality.
8. **Faster Development:** Reduces development time by leveraging existing, well-tested code.
9. **Reduces Duplication:** Prevents code duplication by inheriting common attributes and behaviors.
10. **Promotes Scalability:** Makes it easier to scale applications as new subclasses can be created to extend existing functionality.

Disadvantages of Inheritance

1. **Tight Coupling:** Creates strong dependency between parent and child classes, making the system more rigid.
2. **Fragile Base Class Problem:** Changes in the base class can inadvertently break the functionality of child classes.
3. **Reduced Flexibility:** Forces an "is-a" relationship, which may not always fit the actual design requirement.
4. **Inheritance Misuse:** Can lead to improper design when used only for code reuse instead of representing a true relationship.
5. **Performance Overhead:** Can slow down performance due to deep inheritance hierarchies and method resolution.
6. **Complexity:** Introduces unnecessary complexity when simple composition would suffice.
7. **Limited Modifications:** Modifying the parent class affects all subclasses, which may not always be desirable.
8. **Difficulty in Testing:** Unit testing becomes more challenging due to dependencies between parent and child classes.
9. **Restricted Multiple Inheritance:** Java doesn't support multiple inheritance of classes, limiting design flexibility.
10. **Lack of Control:** Subclasses have access to all public and protected members of the parent class, which can be misused.

Design Patterns

Design patterns are proven, reusable solutions to common software design problems, providing a structured way to address recurring challenges. **Purpose:**

- Simplify complex software design.
- Enhance code reusability, readability, and maintainability.
- Promote best practices and improve system scalability.
- Encourage use of object-oriented principles like encapsulation and polymorphism.

Categories of Design Patterns

1. Creational Patterns: Focus on ways to create objects while hiding the creation logic, ensuring flexibility and decoupling in object creation. **Examples:**

- **Singleton:** Ensures only one instance of a class exists and provides global access to it. Commonly used for logging, configuration management, or database connections.
- **Factory:** Abstracts object creation by returning instances based on input parameters or conditions. Example: Shape Factory (getShape() returns Circle, Square, etc.).
- **Builder:** Constructs complex objects step-by-step using a series of methods. Used when objects have multiple optional components (e.g., building a Car with optional parts).

2. Structural Patterns: Focus on the composition of objects and classes to form larger structures while maintaining flexibility and efficiency. **Examples:**

- **Adapter:** Converts one interface to another, enabling incompatible systems to work together. Example: Adapting legacy system APIs to modern interfaces.
- **Composite:** Treats individual objects and groups of objects uniformly. Example: File system representation (Files and Folders treated similarly).
- **Decorator:** Dynamically adds functionality to objects without modifying their code. Example: Adding formatting options to a text editor (bold, italic).

3. Behavioural Patterns: Focus on interactions and communication between objects, defining how objects collaborate to achieve complex tasks. **Examples:**

- **Observer:** Implements publish-subscribe, where changes in one object notify multiple subscribers. Example: News subscriptions or event listeners.
- **Strategy:** Allows dynamic switching of algorithms at runtime. Example: Choosing different payment methods in an e-commerce app.
- **Command:** Encapsulates a request as an object, allowing for parameterization, queuing, and undo/redo functionality. Example: Implementing a remote control system for devices.

Benefits of Design Patterns

1. Promotes **code reusability** and maintainability.
2. Encourages the use of **best practices**.
3. Enhances collaboration and understanding between developers by providing a standard design vocabulary.
4. Simplifies **scalability** and adaptability in system design.

Iterator Pattern

The **Iterator Pattern** is a behavioural design pattern that provides a way to access elements of a collection **sequentially** without exposing the collection's internal structure. **Purpose:**

- To allow **traversal of a collection** (e.g., lists, sets) without needing to know its internal representation.
- It decouples the logic for accessing elements from the collection's implementation.
- Useful for working with different types of collections in a uniform way.

Aspect	Iterator	Iterable
Definition	An interface used to traverse elements in a collection.	An interface that represents a collection capable of returning an Iterator.
Key Methods	hasNext(), next(), remove()	iterator()
Responsibility	Provides logic for iterating through elements.	Ensures that the collection can produce an Iterator.
Return Type	Does not return itself.	Returns an Iterator instance.
Usage	Used for actual iteration.	Acts as a wrapper for iteration support.
Example	Iterator<String> it = list.iterator();	class MyCollection implements Iterable {}

Java's Collection Framework Implementation of Iterators

- **List:** Collections like ArrayList, LinkedList implement Iterator for sequential traversal. Supports ListIterator for bidirectional traversal.
- **Set:** Classes like HashSet and TreeSet use Iterator for unordered traversal.
- **Map:** Does not directly implement Iterator. Uses keySet(), values(), or entrySet() to create iterators for keys, values, or key-value pairs.
- **Queue:** Classes like PriorityQueue and LinkedBlockingQueue provide Iterator implementations to traverse elements.
- **Fail-Fast Behavior:** Most iterators in Java are **fail-fast**, meaning they throw a ConcurrentModificationException if the collection is modified during iteration.

MODULE 4: MODEL VIEW CONTROLLER PATTERN

Contents

1. Model-View-Controller (MVC) Pattern	1
2. Commands as Methods	2
3. Commands as Objects	2
4. Implementing Object-Oriented Language Features in Java	2

Model-View-Controller (MVC) Pattern

The **MVC pattern** is a software architectural pattern used to build applications by separating the concerns of data, user interface, and user input. It promotes modularity, making the codebase easier to maintain, test, and scale. **Components of MVC**

1. **Model:** Represents the application's **data** and **business logic**. **Responsibilities:**

- Manages the data and application state.
- Performs operations like database access, calculations, and business rule enforcement.
- Notifies the view when the data changes.

Example: In a library app, the Book class can represent a model containing attributes like title, author, and availability.

2. **View:** Represents the **user interface** (UI) of the application. **Responsibilities:**

- Displays the data from the model to the user.
- Listens for updates from the model and refreshes the UI.
- Does not directly interact with the model.

Example: In a GUI application, the JTable or TextField components displaying book data act as the view.

3. **Controller:** Acts as an intermediary between the **view** and the **model**, handling user inputs and triggering updates. **Responsibilities:**

- Processes user input from the view.
- Updates the model based on user actions.
- Notifies the view to refresh its display after changes in the model.

Example: In the library app, clicking a "Borrow Book" button triggers the controller to update the book's availability in the model.

4. **Separation of Concerns:** Each component (Model, View, Controller) focuses on a specific concern, improving modularity. **Benefits:**

- Maintainability:** Changes to the UI (view) don't affect business logic (model) and vice versa.
- Scalability:** Easier to add features or modify individual components.
- Testability:** Components can be tested independently.

Example: If a new user interface is required (e.g., switching from Swing to JavaFX), the view can be replaced without altering the model or controller.

Flow in MVC

- User interaction:** The user interacts with the view (e.g., clicks a button).
- Controller processes input:** The controller processes the input and updates the model.
- Model changes state:** The model updates its data and notifies the view.
- View updates:** The view reflects the changes in the model.

Commands as Methods

- **Definition:** Actions or operations are encapsulated as individual methods within classes.
- **Features:**
 - Direct execution of commands through method calls.
 - Suitable for small, simple applications where the action logic is straightforward.
- **Benefits:**
 - Simplicity: Easy to implement and understand.
 - Minimal overhead: No need for additional structures like objects.
- **Drawbacks:**
 - Tight coupling: The caller and the receiver are closely linked, making it less flexible for complex applications.
 - Difficult to extend: Adding new commands requires modifying the existing code.

Commands as Objects

- **Definition:** Encapsulates requests as separate objects, decoupling the sender (invoker) and the receiver (executor). This is a core concept of the **Command Pattern**.
- **Features:**
 - Each command is represented as a class implementing a common interface (e.g., Command with execute()).
 - Enables queuing, logging, undo/redo operations, and parameterization of commands.
- **Benefits:**
 - Decouples sender and receiver, improving flexibility and scalability.
 - Supports advanced features like undo/redo.
- **Drawbacks:**
 - Increased complexity: More classes and structures are needed.
 - Slightly higher memory overhead due to object creation.

Aspect	Commands as Methods	Commands as Objects
Definition	Encapsulates actions as methods.	Encapsulates requests as objects.
Complexity	Simple and minimal overhead.	More complex due to multiple classes.
Flexibility	Less flexible; tightly coupled.	Highly flexible; decouples sender/receiver.
Scalability	Limited for large applications.	Scales well for advanced functionality.
Use Case	Small and straightforward operations.	Queuing, logging, undo/redo functionality.

Implementing Object-Oriented Language Features in Java

Java-Specific Features

1. **Inner Classes (Static and Non-Static):** Inner classes are classes defined within another class, useful for logically grouping classes or for implementation of event listeners.

Types:

- **Non-static Inner Classes:** Have access to instance variables of the outer class.
- **Static Nested Classes:** Can be instantiated without an instance of the outer class.

```
class Outer {  
    private int data = 10;  
  
    class Inner {  
        void display() {  
            System.out.println("Data: " + data);  
        }  
    }  
  
    static class StaticNested {  
        void show() {  
            System.out.println("Static nested class");  
        }  
    }  
}
```

2. Anonymous Classes and Lambda Expressions

- **Anonymous Classes:** Classes with no name, defined in a single expression, often used for implementing interface methods.

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running in a thread");  
    }  
};  
Thread thread = new Thread(r);  
thread.start();
```

- **Lambda Expressions:** Concise way to express instances of functional interfaces (interfaces with a single abstract method).

```
// Using Lambda to implement Runnable  
Runnable r = () -> System.out.println("Running with lambda!");  
Thread thread = new Thread(r);  
thread.start();
```

3. Functional Interfaces (Consumer, Predicate, etc.): An interface with a single abstract method, used with lambda expressions. **Common Functional Interfaces:**

- **Consumer:** Takes a single input and performs an action (e.g., `forEach()`).
- **Predicate:** Takes a single input and returns a boolean.
- **Function:** Takes an argument and returns a result.

```
// Consumer Example  
Consumer<String> print = (s) -> System.out.println(s);  
print.accept("Hello, World!");  
  
// Predicate Example  
Predicate<Integer> isEven = (x) -> x % 2 == 0;  
System.out.println(isEven.test(4)); // true
```

4. Generics (e.g., `List<T>` and Custom Generic Classes):

Generics allow you to create classes, methods, and interfaces that can operate on objects of various types while providing compile-time type safety. **How to Implement:** Use type parameters (e.g., T, E, K, V) in class, interface, or method declarations.

```
// Generic class  
class Box<T> {  
    private T value;  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
}  
  
// Using the generic class  
Box<Integer> integerBox = new Box<>();  
integerBox.setValue(10);  
System.out.println(integerBox.getValue()); // 10
```

MODULE 5.1: JAVA GENERICS

Contents

1. Java Generics	1
2. Advantages of Generics	2
3. Disadvantages of Generics	2
4. Applications of Generics	4
5. Type Erasure	4
6. Java's Collections Framework	5
7. ArrayList	6
8. When to Use ArrayList	6

Java Generics

Java Generics allow you to create classes, interfaces, and methods that can operate on any specified type without needing to define them for each object type. Generics help achieve type safety and reduce redundancy, making the code more reusable and flexible.

Generic Classes: A generic class is a class that can operate on objects of various types while providing compile-time type safety. It allows you to define a class with a type parameter that can be specified when creating an instance of the class.

Generic Method: are methods that introduce their own type parameters. This allows you to create methods that can operate on different types of data.

```
public <T> void methodName(T parameter) {  
    // Method implementation  
}
```

```
class ClassName<T> {  
    // Member variables  
    private T data;  
  
    // Constructor  
    public ClassName(T data) {  
        this.data = data;  
    }  
  
    // Methods  
    public T getData() {  
        return data;  
    }  
  
    public void setData(T data) {  
        this.data = data;  
    }  
}
```

Key Points in Generic Class and Methods

1. **Type Parameters:** In both generic classes and methods, type parameters (e.g., T, E) can be any identifier, and you can use multiple type parameters (e.g., <T, U>).
2. **Instantiation:** You can create instances of generic classes with specific types (e.g., Box<Integer>), ensuring type safety.
3. **Type Inference:** Java can often infer the type parameters when you instantiate a generic class or call a generic method.
4. **Bounded Type Parameters:** You can restrict the types that can be passed to generic classes or methods using bounds (e.g., class Box<T extends Number>).
5. **Static Members:** Static members of a generic class cannot use the type parameters, but they can use a specific type.

Advantages of Generics

1. **Type Safety:** Ensures that only the specified data type is used, reducing runtime errors.
2. **Code Reusability:** Allows developers to create reusable code components that work with any object type.
3. **Elimination of Casting:** Reduces the need for explicit type casting, simplifying code and improving readability.
4. **Stronger Type Checks:** Provides compile-time checks, catching type-related errors earlier in the development process.
5. **Generic Algorithms:** Facilitates the development of algorithms that can operate on various data types without code duplication.
6. **Improved Performance:** Reduces the overhead of type casting, leading to better performance.
7. **Easier Maintenance:** Code is cleaner and easier to maintain due to less boilerplate code.
8. **Support for Collections:** Enhances the Java Collections Framework, allowing for type-safe collections.
9. **Flexibility:** Enables the creation of data structures that can store any object type.
10. **Interoperability:** Allows for more seamless integration with existing Java APIs and libraries.

Disadvantages of Generics

1. **Complexity:** Can add complexity to code, making it harder for beginners to understand.
2. **Limited Support for Primitive Types:** Only reference types can be used; primitives must be wrapped in their corresponding wrapper classes.
3. **Type Erasure:** At runtime, generic type information is erased, which can limit some functionality.
4. **Cannot Create Generic Arrays:** Creating arrays of generic types is not allowed, leading to workarounds.
5. **Incompatibility with Older Code:** May complicate interactions with legacy code that doesn't use generics.
6. **No Primitive Types:** Generics do not support primitive types (e.g., int, char). Instead, you must use their wrapper classes (e.g., Integer, Character).
7. **Raw Types:** Using raw types (unparameterized generics) is allowed but discouraged, as it bypasses type safety and can lead to runtime errors.
8. **Wildcards Confusion:** Use of wildcards can be confusing and lead to complex code structures.
9. **Compile-Time Restrictions:** Some type restrictions may hinder flexibility in certain scenarios.
10. **Requires Careful Design:** Generics must be designed thoughtfully to avoid misuse and maintain readability.
11. **Limited Type Information:** Lack of runtime type information can lead to issues with reflection.
12. **Overhead for Some Scenarios:** In certain cases, using generics may introduce overhead that affects performance.

```

import java.util.ArrayList;
import java.util.*;
//Generic class with two type parameters: T1 and T2
class MyGeneric<T1, T2>{
    private int val;
    private T1 t1;
    private T2 t2;
    //Constructor for MyGeneric class
    public MyGeneric(int val, T1 t1, T2 t2) {
        this.val = val;
        this.t1 = t1;
        this.t2 = t2;
    }
    //Getter & Setter for T1
    public T1 getT1() {return t1;}
    public void setT1(T1 t1) {this.t1 = t1;}
    //Getter & Setter for T2
    public T2 getT2() { return t2;}
    public void setT2(T2 t2) {this.t2 = t2;}
    //Getter & Setter for val
    public int getVal() {return val;}
    public void setVal(int val) {this.val = val;}
}

public class GenericsBox{
    public static void main(String[] args) {
        ArrayList<Integer> arrayList = new ArrayList<>();

        //Creating a MyGeneric object with String and Integer types
        //val=21,t1=Triyasha Das,t2=45
        MyGeneric<String, Integer> g1 = new MyGeneric<>(23, "Parthajit Das", 45);

        //Retrieving values from the generic object g1
        String str = g1.getT1();
        Integer int1 = g1.getT2();
        System.out.println("String: " + str + ", Integer: " + int1);

        //Additional usage examples of generics
        //val=100, t1=123.45,t2= true
        MyGeneric<Double, Boolean> g2 = new MyGeneric<>(100, 123.45, true);

        Double d= g2.getT1();
        Boolean b= g2.getT2();
        System.out.println("Double: " + d + ", Boolean: " + b);
    }
}

```

BlueJ: Terminal Window - PARTHAJIT

Options

String: Parthajit Das, Integer: 45
Double: 123.45, Boolean: true

Applications of Generics

1. **Collections Framework:** Used extensively in Java's Collection classes (e.g., List, Set, Map).
2. **Custom Data Structures:** Enables the creation of type-safe custom data structures like stacks, queues, and trees.
3. **Generic Algorithms:** Facilitates the development of algorithms that can work with different data types.
4. **Type-safe APIs:** Helps create APIs that ensure type safety, reducing errors in client code.
5. **Sorting and Searching:** Enables generic methods for sorting and searching that work with various data types.
6. **Functional Interfaces:** Commonly used in functional interfaces for lambda expressions and streams.
7. **Service-Oriented Architecture:** Supports the development of generic services that can handle different data types.
8. **Framework Development:** Used in building frameworks that require flexibility with data types, like Spring.
9. **Database Operations:** Helps in creating generic data access objects (DAOs) that work with different entities.
10. **Testing:** Facilitates the creation of generic test cases that can be reused across different data types in unit testing.

Type Erasure

Type Erasure is a fundamental concept in Java generics that occurs at runtime. It refers to the process by which the Java compiler removes all information about generic types after type checking is complete. This allows for backward compatibility with legacy code that does not use generics.

1. **Removal of Type Parameters:** Generic type parameters are replaced with their bounds or Object after compilation.
2. **Loss of Generic Type Information:** Type information is not available at runtime, limiting certain type-related operations.
3. **Automatic Casting:** The compiler adds necessary casts to convert Object back to the specific type when retrieving items.
4. **Bounded Types:** Bounds are replaced with the first bound type; additional bounds are ignored.
5. **Static Members:** Static members cannot use type parameters directly.
6. **Raw Types:** Raw types (unparameterized generic types) are allowed but bypass type safety.

```
class GenericBox<T> {  
    private T item;  
    public GenericBox(T item) {  
        this.item = item;  
    }  
    public T getItem() {  
        return item;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        GenericBox<Integer> box = new GenericBox<>(123);  
        Integer value = box.getItem();  
        // No cast needed; it's safe due to generics  
    }  
}
```

After Type Erasure, the GenericBox class would look like this:

```
class GenericBox {  
    private Object item;  
    public GenericBox(Object item) {  
        this.item = item;  
    }  
    public Object getItem() {  
        return item;  
        // Requires casting to retrieve the specific type  
    }  
}
```

Implications of Type Erasure

1. **Runtime Type Information Loss:** No access to generic type information at runtime, limiting reflection capabilities.
2. **Backward Compatibility:** Ensures generics work with legacy code that predates their introduction.
3. **Type Safety Limitations:** Potential for runtime exceptions due to type casting.
4. **No Performance Overhead:** Since type information is erased, there's no runtime performance cost associated with generics.
5. **Limited Operations:** Certain operations, like instanceof, cannot be performed on generic types at runtime.

Java's Collections Framework

Java's Collections Framework is a unified architecture that provides data structures and algorithms for storing and manipulating groups of objects. It offers various interfaces, implementations, and algorithms to handle collections of data efficiently.

1. **Generics:** Collections are strongly typed using generics, ensuring type safety and reducing runtime errors.
2. **Iterators:** The Iterator interface provides a way to traverse collections without exposing their underlying structure.
3. **Concurrent Collections:** Java provides thread-safe collection implementations (e.g., ConcurrentHashMap, CopyOnWriteArrayList) for concurrent programming.
4. **Utility Methods:** The Collections class provides static methods for operations such as sorting, searching, and synchronizing collections.
5. **Stream API:** The Collections Framework is integrated with the Stream API, allowing for functional-style operations on collections.

Key Interfaces

1. **Collection:** The root interface for the collection hierarchy. It defines basic methods for handling groups of objects.
2. **List:** An ordered collection (also known as a sequence) that allows duplicate elements. Key implementations include:
 - a. **ArrayList:** A resizable array implementation, offering fast random access.
 - b. **LinkedList:** A doubly linked list implementation, suitable for frequent insertions and deletions.
3. **Set:** A collection that does not allow duplicate elements. Key implementations include:

- a. **HashSet**: Uses a hash table for storage; offers constant-time performance for basic operations.
 - b. **LinkedHashSet**: Maintains insertion order while using a hash table.
 - c. **TreeSet**: Implements a sorted set, storing elements in natural order or by a specified comparator.
- 4. **Map**: An object that maps keys to values, with no duplicate keys allowed. Key implementations include:
 - a. **HashMap**: Uses a hash table; provides constant-time performance for basic operations.
 - b. **LinkedHashMap**: Maintains insertion order while using a hash table.
 - c. **TreeMap**: Implements a sorted map, storing key-value pairs in natural order or by a specified comparator.
- 5. **Queue**: A collection used to hold elements prior to processing. Key implementations include:
 - a. **PriorityQueue**: Orders elements based on their natural ordering or a provided comparator.
 - o **ArrayDeque**: A resizable array implementation of the deque interface.

Advantages

1. **Ease of Use**: Provides ready-to-use data structures and algorithms.
2. **Performance**: Optimized implementations for different use cases.
3. **Flexibility**: Supports various data structures, allowing developers to choose the best one for their needs.
4. **Interoperability**: Works seamlessly with Java's other core libraries and features.

ArrayList

ArrayList is a part of Java's Collections Framework and provides a resizable array implementation of the List interface. It allows for dynamic resizing, meaning the size of the ArrayList can grow or shrink as needed when elements are added or removed.

Key Features of ArrayList

1. **Dynamic Size**: Unlike arrays, which have a fixed size, ArrayList can grow as elements are added, allowing for flexible data storage.
2. **Ordered Collection**: Elements in an ArrayList maintain the order of insertion, and you can access them using their index.
3. **Allows Duplicates**: ArrayList can store duplicate elements, meaning you can add the same value multiple times.
4. **Fast Random Access**: Provides $O(1)$ time complexity for accessing elements using an index due to its underlying array structure.
5. **Performance**: While adding elements at the end of the list is generally $O(1)$, adding elements in the middle or beginning can be $O(n)$ due to shifting.

When to Use ArrayList

1. **Dynamic Sizing**: You need a resizable array that can grow or shrink as elements are added or removed.
2. **Fast Random Access**: You require fast access to elements by index ($O(1)$ time complexity).
3. **Ordered Collection**: You need to maintain the insertion order of elements.
4. **Duplicates Allowed**: You want to store duplicate elements.
5. **Frequent Reads**: Your use case involves frequent retrievals (`get()`) and relatively fewer insertions or deletions.

6. **Sequential Inserts at the End:** You primarily insert elements at the end of the list, which is efficient.
7. **When Thread Safety is Not Required:** You don't need a thread-safe collection.
8. **Low-Memory Footprint:** You want a relatively compact data structure for storing elements.
9. **When the Number of Insertions/Deletions is Minimal:** You don't need frequent modifications (additions/removals) in the middle of the list.
10. **Need for a General-Purpose List:** You need a flexible and widely used list implementation for general purposes.

Common Methods

Here are some commonly used methods in the `ArrayList` class:

- `add(E e)`: Adds an element to the end of the list.
- `add(int index, E element)`: Inserts an element at a specified position in the list.
- `remove(int index)`: Removes the element at the specified position.
- `remove(Object o)`: Removes the first occurrence of the specified element.
- `get(int index)`: Returns the element at the specified position.
- `set(int index, E element)`: Replaces the element at the specified position with the specified element.
- `size()`: Returns the number of elements in the list.
- `clear()`: Removes all elements from the list.
- `contains(Object o)`: Checks if the list contains the specified element.
- `indexOf(Object o)`: Returns the index of the first occurrence of the specified element, or -1 if not found.
- `toArray()`: Converts the list to an array.

MODULE 5.2: JAVA GUI SWING AND SCALE

Contents

1. Abstract Window Toolkit.....	1
2. Java Swing.....	2
3. Swing vs. AWT.....	2
4. Basic Structure of a Swing Application.....	3
5. Advantages of swing class.....	6
6. Disadvantages of Swing	6
7. Swing Component Hierarchy	7
8. Event Handling in Swing.....	8
9. ActionListeners and Other Listeners in Swing.....	9
10. ActionEvent and Processing with ActionListener	11

Abstract Window Toolkit

AWT (Abstract Window Toolkit) is a Java package that provides the ability to create graphical user interfaces (GUIs) in Java applications. `java.awt.*`; package provides classes like Button, Frame, Canvas, Color, Choice etc. **Key Features of AWT**

1. **Heavyweight Components:** AWT components are heavyweight because they rely on the native operating system's graphical user interface. This means that they are rendered by the OS rather than by Java itself.
2. **Platform Dependency:** Since AWT components are tied to the native system, their appearance and behaviour can vary between different operating systems.
3. **Event Handling:** AWT supports event-driven programming, allowing developers to handle user interactions (like clicks and key presses) through event listeners.
4. **Basic GUI Components:** AWT provides fundamental GUI components such as buttons, labels, text fields, checkboxes, and menus.

Common AWT Components

1. **Frame:** A top-level window with a title and borders.
2. **Button:** A clickable button that performs an action when pressed.
3. **Label:** A non-editable text display component.
4. **TextField:** A single-line input field for text.
5. **TextArea:** A multi-line input field for text.
6. **Checkbox:** A box that can be checked or unchecked.
7. **List:** A list of items from which users can select.
8. **Menu:** A dropdown menu containing various options.

Advantages of AWT

1. **Simplicity:** AWT has a simpler API compared to other GUI frameworks, making it easier for beginners to use.

2. **Native Look and Feel:** AWT components provide a native look and feel on different operating systems, as they use the underlying system's UI.
3. **Lightweight Footprint:** AWT applications tend to use fewer resources compared to more complex frameworks.

Disadvantages of AWT

1. **Heavyweight Components:** AWT components can be slower and consume more memory due to their dependence on native resources.
2. **Limited Components:** AWT provides a basic set of components compared to other GUI frameworks like Swing or JavaFX.
3. **Inconsistent Behaviour:** Because AWT relies on the underlying OS, the appearance and behaviour of components may vary across platforms.
4. **Limited Customization:** AWT components are less customizable compared to Swing components, which support a pluggable look-and-feel.

Java Swing

Swing is a part of the Java Foundation Classes (JFC) library and serves as an extension of the Abstract Window Toolkit (AWT) for creating windowed applications in Java. It allows us to use more tools on a simple and lightweight platform, extend the functionality, and improve the result by supporting drag and drop. Swing is built on the top of AWT API and entirely written in Java.

Java Swing provides a simple way to create graphical user interfaces, making it easy for beginners to use and very effective for experienced developers. This simple application introduces the basics of creating GUIs with Swing and can be a stepping stone to creating more interactive and intuitive applications.

javax.swing package provides classes such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser, etc. for the Java swing API.

1. **Lightweight Components:** Swing components are written entirely in Java and do not rely on the underlying platform's windowing system, making them platform-independent.
2. **Rich Set of GUI Components:** Swing provides a comprehensive set of GUI components, including buttons, checkboxes, radio buttons, tables, trees, and more.
3. **Pluggable Look and Feel:** Swing supports pluggable look-and-feel designs, allowing developers to customize the appearance of GUI widgets/applications(Look) and the way they behave(Feel) to match different platforms or custom themes.
4. **Event Handling:** Swing follows a robust event-handling model, making it easy to manage user interactions.
5. **MVC Architecture:** Swing components follow the Model-View-Controller (MVC) design pattern, separating the data (model) from the user interface (view) and the logic that controls the interaction (controller).
6. **Highly Customizable Components:** Swing components can be customized and extended easily to create rich user interfaces.

Swing vs. AWT

Feature	Swing	AWT
Component Type	Components are platform independent	Components are platform dependent

Component Handling	Uses lightweight components that are drawn in Java, not dependent on the underlying OS	Uses heavyweight components tied to the native OS
Look-and-Feel	Supports pluggable look-and-feel; can be customized easily	Limited to the native look-and-feel of the OS
Event Handling	More advanced event handling capabilities	Basic event handling
Flexibility and components	Highly customizable; offers more components and layout managers	Less flexible; fewer components available
Performance	Generally slower due to rendering overhead	Faster due to direct access to native resources
Complexity	More complex due to a richer feature set	Simpler API with fewer features
MVC support	Swing follows MVC	AWT doesn't follow MVC where model represents data, view represents presentation and controller acts as an interface between model and view
Accessibility	Better support for accessibility features	Limited support for accessibility

Basic Structure of a Swing Application

1. Import Required Packages
2. Create the Main Application Class
3. Create and configure the JFrame
4. Create and Add Components to the JFrame
5. Implement Event Handling
6. Set the JFrame Visibility

1. **Import Required Packages:** In Java, delivery packages are required so that we can use the classes and interactions in these packages according to our needs in our program. Packages are used to group related and interactive resources together to help us plan and develop policies. For the Swing application here, we are importing the following packages:

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

- a. javax.swing: provides a set of 'lightweight' (all-Java language) components that, unlike the AWT components, are platform-independent and can be customised easily. Swing includes components such as buttons, text fields, labels, tables, trees, and more.
 - i. java.awt.event.ActionEvent:
 - ii. The java.awt.event package provides interfaces and classes for dealing with different types of events fired by AWT and Swing components.
 - iii. ActionEvent is a class that represents the occurrence of an action. It is generated when a component-defined action occurs, such as when a button is clicked.
- b. import java.awt.event.ActionListener;

- i. The `java.awt.event.ActionListener` interface is used to receive action events.
 - ii. The interface defines a single method, `actionPerformed(ActionEvent e)`, which is called when an action event is fired. In this case, it is used to handle the button click event.
2. **Create the Main Application Class:** The main class of a Java Swing application is the implementation of the first application. This class usually has a main method that sets the application's window size and adds necessary properties to it. The main applications in the program are:

```
public class BasicSwingApp {
```

3. **Create and Configure the JFrame:** `JFrame` is the main window of a Java Swing application. It acts as a container for all other objects such as panels, buttons, labels, and text. The `JFrame` used here is a window with decorations such as a title, border, and buttons for closing and minimizing.

```
JFrame frame = new JFrame("Basic Swing Application");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 200);
```

4. **Create and Add Components to the JFrame:** Creating and adding components to a `JFrame` involves several steps, including initializing the frame, creating objects (such as buttons, labels, tags), and adding these objects to the frame details pane.

```
JPanel panel = new JPanel();
frame.add(panel);
placeComponents(panel);
```

5. **Implement Event handling:** Using event handlers in a Swing application involves adding listeners to objects (such as buttons) that can trigger events. You should also set the `JFrame` to visible at the end of the configuration to ensure that the window is displayed while the application is running.

```
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // Get the text from the JTextField
        String text = textField.getText();

        // Show a message dialog with the entered text
        JOptionPane.showMessageDialog(null, "You entered: " + text);
    }
});
```

6. **Set the JFrame visibility:** It is important to set the `JFrame` view to display the window while your application is running. This is usually done at the end of the `JFrame` configuration to ensure that all components are added and configured correctly before displaying the window.

```
frame.setVisible(true);
```

```

package sem5;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class BasicSwingApp {
    public static void main(String[] args) {
        // Create and configure the JFrame
        JFrame frame = new JFrame("Basic Swing Application");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 200);

        // Create a JPanel and add components
        JPanel panel = new JPanel();
        frame.add(panel);
        placeComponents(panel);

        // Set the JFrame's visibility
        frame.setVisible(true);
    }
    private static void placeComponents(JPanel panel) {
        panel.setLayout(null);

        // Create a JLabel
        JLabel label = new JLabel("Enter text:");
        label.setBounds(10, 20, 80, 25);
        panel.add(label);

        // Create a JTextField
        JTextField textField = new JTextField(20);
        textField.setBounds(100, 20, 165, 25);
        panel.add(textField);

        // Create a JButton
        JButton button = new JButton("Show Message");
        button.setBounds(10, 80, 150, 25);
        panel.add(button);

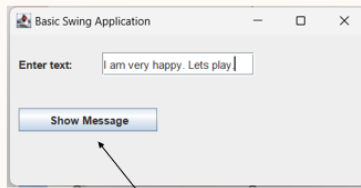
        // Implement event handling for the JButton
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Get the text from the JTextField
                String text = textField.getText();

                // Show a message dialog with the entered text
                JOptionPane.showMessageDialog(null, "You entered: " + text);
            }
        });
    }
}

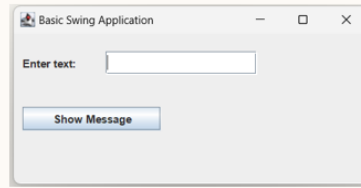
```

OUTPUT

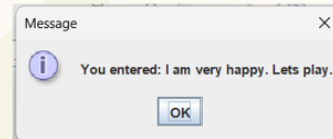
Step 1: After the compilation, we get this screen below where we need to enter a text in the blank



Step 3: We get a response popup where our entered text is displayed and a “OK” button for an exit.



Step 2: We write a random message and click on the “Show Message” Button.



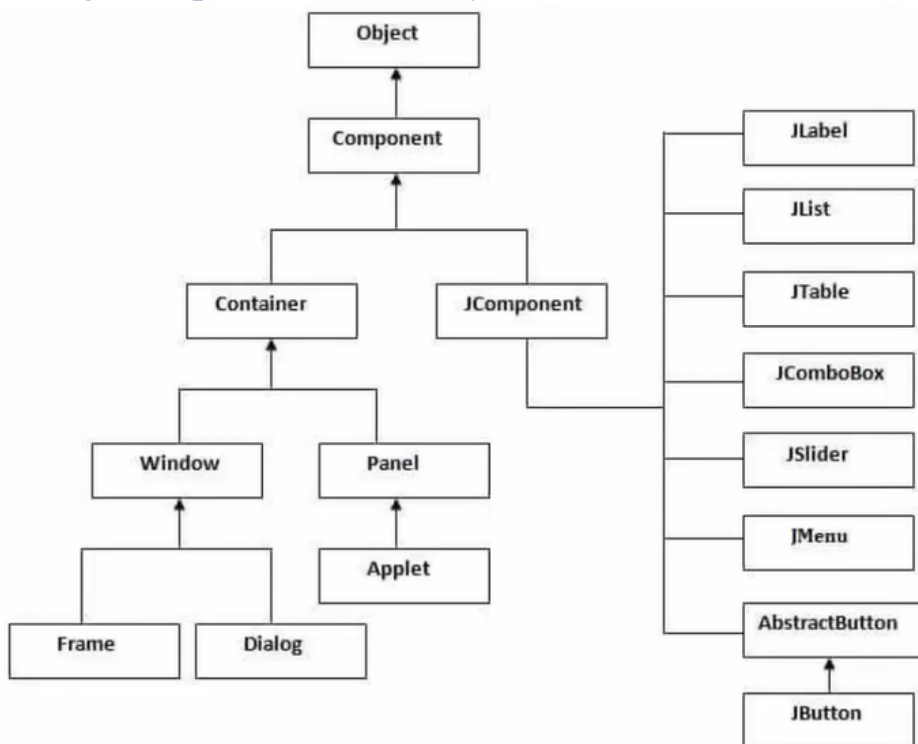
Advantages of swing class

1. **Platform Independence:** Swing components are written entirely in Java, making them platform-independent. This ensures consistent behavior and appearance across different operating systems.
2. **Rich Set of Components:** Swing offers a comprehensive set of GUI components, such as buttons, checkboxes, radio buttons, tables, trees, text fields, and more. These components are highly customizable.
3. **Pluggable Look-and-Feel:** Swing supports the pluggable look-and-feel (PLAF) architecture, allowing developers to change the appearance of applications dynamically. Applications can look like native applications on different platforms or have a custom look.
4. **Customizability:** Swing components can be easily customized. Developers can subclass existing components or create new ones to meet specific requirements.
5. **Lightweight Components:** Unlike AWT (Abstract Window Toolkit), Swing components are lightweight, meaning they are not dependent on native peers. This results in more efficient use of resources and better performance.
6. **Advanced Features:** Swing provides advanced features such as drag-and-drop support, double buffering, and a powerful text component framework that supports formatted text, hyperlinks, and more.
7. **Extensibility:** Swing is highly extensible. Developers can extend existing components or create new components with custom behavior.

Disadvantages of Swing

1. **Performance:** Swing applications can be slower compared to native applications due to additional overhead.
2. **Look and Feel:** Swing may not provide a consistent native look and feel across different platforms, leading to a less integrated user experience.
3. **Complexity:** Building complex UIs can become cumbersome with Swing, requiring more code and design effort.
4. **Heavyweight:** Swing components can consume more memory than lightweight components, affecting resource usage.
5. **Limited Accessibility:** Swing does not have built-in support for some accessibility features, making it harder for users with disabilities to interact with applications.

Swing Component Hierarchy



1. **Top-Level Containers:** These are the primary containers that hold other components. Every Swing application must have at least one top-level container.
 - a. **JFrame:** Represents a window with standard decorations (title bar, borders, etc.). It is commonly used for main application windows.
 - b. **JDialog:** A pop-up window used for secondary tasks. Can be modal (blocking input to other windows) or non-modal.
 - c. **JApplet:** A Swing version of an applet, which is embedded in web browsers. (Less commonly used now due to the decline of applets.)
 - d. **JLayeredPane:** Allows stacking of components in layers, providing greater control over the display order.
 - e. **JRootPane:** The base class for Swing top-level containers, providing a common structure for JFrame and JDialog.
2. **Root Component:** **JComponent** is the base class for all Swing components except for the top-level containers. It provides common methods for painting, event handling, and borders. All Swing components, like buttons, labels, and panels, inherit from JComponent.
3. **Containers:** These components are used to organize and group other components.
 - a. **JPanel:** A generic container for grouping components, often used to create layouts.
 - b. **JScrollPane:** Provides a scrollable view for other components like JTextArea or JList.
 - c. **JTabbedPane:** Allows switching between different panels (tabs) within the same window.
 - d. **JSplitPane:** Divides two components, allowing the user to resize them.
 - e. **JToolBar:** A container for grouping tool buttons and controls.

4. **Basic Swing Components:** These are the most used components for building user interfaces.
 - a. **JButton:** A clickable button.
 - b. **JLabel:** Displays non-editable text or images.
 - c. **JTextField:** A single-line text field for user input.
 - d. **JTextArea:** A multi-line text area for input.
 - e. **JList:** Displays a list of items for selection.
 - f. **JComboBox:** A drop-down list that allows selection from a list of options.
 - g. **JCheckBox:** A box that can be checked or unchecked.
 - h. **JRadioButton:** A button in a group where only one can be selected at a time.
5. **Specialized Components:**
 - a. **JSpinner:** Allows users to select a value from a fixed set or within a range.
 - b. **JProgressBar:** Displays progress for ongoing tasks.
 - c. **JSlider:** A control for selecting a value within a range by sliding a knob.
6. **Layout Managers:** Swing components rely on layout managers to organize the components within containers. Common layout managers include:
 - a. **BorderLayout:** Divides the container into five areas: North, South, East, West, and Center.
 - b. **FlowLayout:** Places components in a row, wrapping to the next line as needed.
 - c. **GridLayout:** Arranges components in a grid of specified rows and columns.
 - d. **BoxLayout:** Aligns components either vertically or horizontally.
7. **Event Handling:** Components generate events that can be handled by implementing listeners. Common listener interfaces include:
 - a. **ActionListener:** For handling button clicks and other actions.
 - b. **MouseListener:** For mouse events (clicks, enters, exits).
 - c. **KeyListener:** For keyboard events.
8. **Lists, Tables, and Trees:**
 - a. **JList:** Displays a list of items.
 - b. **JTable:** Displays data in tabular form.
 - c. **JTree:** Displays hierarchical data.

Event Handling in Swing

Event handling in Swing is a crucial aspect that allows applications to respond to user interactions, such as mouse clicks, keyboard inputs, and other actions. Swing operates on an **event-driven programming** model, where the flow of the program is determined by events generated by the user or the system.

1. **Event-Driven Programming:** is a programming paradigm where the flow of the program is determined by events—actions or occurrences that are detected by the program. These events can be triggered by user actions or other sources.
2. **How It Works:** In Swing, when a user interacts with a GUI component (like clicking a button or typing in a text field), an **event** is generated. This event is then dispatched to the appropriate event handler.
3. **Components of Event Handling:**

- a. **Event Source:** This is the component that generates an event. For example, a button is an event source when it is clicked.
 - b. **Event Object:** When an event occurs, an event object is created that contains information about the event. For instance, a `MouseEvent` contains details about the mouse click, such as its position and the button clicked.
 - c. **Event Listener:** This is an interface that defines methods to handle specific types of events. For example, the `ActionListener` interface is used to handle action events (like button clicks).
4. **Event Registration:** To handle events, you need to register an event listener with the event source. This is done using the `add` method provided by the component. For instance, to add an `ActionListener` to a button, you would use:

```
button.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // Code to handle the button click  
    }  
});
```

5. **Event Dispatching:** When an event occurs, the event dispatching thread (EDT) sends the event to the registered listener. The listener then executes the appropriate method to handle the event. In Swing, all event handling occurs on the EDT to ensure thread safety and proper rendering of the GUI.
6. **Types of Events:** Swing supports various types of events, including:
- i. **Action Events:** Triggered by actions such as button clicks or menu selections. Handled by the `ActionListener` interface.
 - ii. **Mouse Events:** Triggered by mouse actions (click, press, release, enter, exit). Handled by the `MouseListener` and `MouseMotionListener` interfaces.
 - iii. **Key Events:** Triggered by keyboard actions (key press, key release). Handled by the `KeyListener` interface.
 - iv. **Window Events:** Triggered by actions related to window management (opening, closing, iconifying). Handled by the `WindowListener` interface.

ActionListeners and Other Listeners in Swing

In Swing, listeners are interfaces that define methods to handle specific types of events.

Implementing these interfaces allows your application to respond to user actions, such as clicking a button, moving the mouse, typing on the keyboard, or interacting with windows.

1. **ActionListener:** Handles action events, such as button clicks. **Methods:**
 - i. `void actionPerformed(ActionEvent e)`: Invoked when an action occurs.

```

JButton button = new JButton("Submit");
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
});

```

2. **MouseListener:** Handles mouse events (clicks, presses, releases, enters, exits). **Methods:**

- i. void mouseClicked(MouseEvent e)
- ii. void mousePressed(MouseEvent e)
- iii. void mouseReleased(MouseEvent e)
- iv. void mouseEntered(MouseEvent e)
- v. void mouseExited(MouseEvent e)

```

JPanel panel = new JPanel();
panel.addMouseListener(new MouseListener() {
    @Override
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked at: " + e.getX() + ", " + e.getY());
    }

    // Other methods can be left empty or implemented as needed
});

```

3. **KeyListener:** Handles keyboard events (key presses and releases). **Methods:**

- i. void keyTyped(KeyEvent e)
- ii. void keyPressed(KeyEvent e)
- iii. void keyReleased(KeyEvent e)

```

JTextField textField = new JTextField();
textField.addKeyListener(new KeyListener() {
    @Override
    public void keyPressed(KeyEvent e) {
        System.out.println("Key pressed: " + e.getKeyChar());
    }

    // Other methods can be left empty or implemented as needed
});

```

4. **WindowListener:** Handles window events (opening, closing, iconifying, etc.). **Methods:**

- i. void windowOpened(WindowEvent e)
- ii. void windowClosing(WindowEvent e)
- iii. void windowClosed(WindowEvent e)
- iv. void windowIconified(WindowEvent e)
- v. void windowDeiconified(WindowEvent e)
- vi. void windowActivated(WindowEvent e)
- vii. void windowDeactivated(WindowEvent e)

```

JFrame frame = new JFrame("Window Example");
frame.addWindowListener(new WindowListener() {
    @Override
    public void windowClosing(WindowEvent e) {
        System.out.println("Window is closing!");
        frame.dispose();
    }

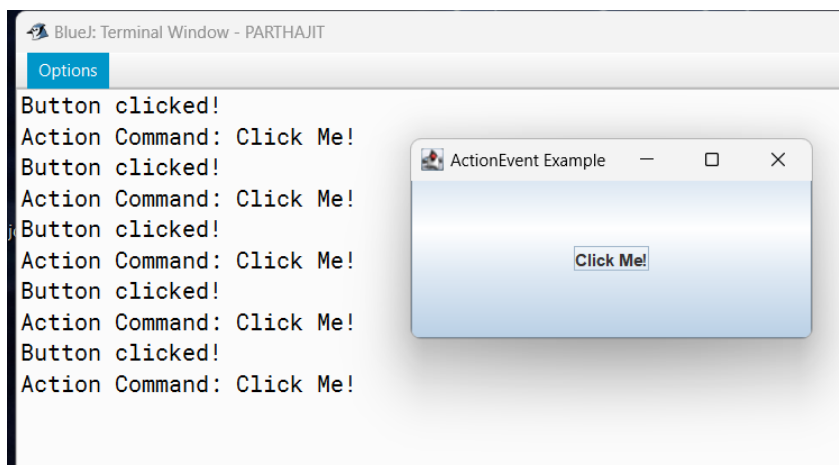
    // Other methods can be left empty or implemented as needed
});

```

ActionEvent and Processing with ActionListener

1. **ActionEvent** is a class in the java.awt.event package that represents an action event, such as a button click. It contains information about the event, including the source component (the button that was clicked).
2. **Creating ActionEvent:** An ActionEvent is automatically created when an action occurs, such as clicking a button.
3. **Processing ActionEvents:**
 - a. To process ActionEvent, you implement the ActionListener interface and define the actionPerformed method.
 - b. The ActionEvent object passed to actionPerformed contains information about the event, such as the command string, source, and modifiers.

Output



```

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class ActionEventExample {
    public static void main(String[] args) {
        // Create a JFrame
        JFrame frame = new JFrame("ActionEvent Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 150);

        // Create a JButton
        JButton button = new JButton("Click Me!");

        // Add ActionListener to the button
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Handle the button click
                System.out.println("Button clicked!");
                // Get additional info from ActionEvent
                System.out.println("Action Command: " + e.getActionCommand());
            }
        });

        // Add button to the frame
        frame.getContentPane().add(button);
        frame.setVisible(true);
    }
}

```